程序设计思维与实践

课程笔记

作者　浮萍　　版本　v1.2

把类型、运算与结构想清楚，代码才会替你说话。

QQ 邮箱　3071974740@qq.com

GitHub　github.com/Duckweed-yhb

本笔记为个人学习整理，内容仍在持续完善。如发现内容、公式、文字或排版问题，欢迎通过上述 QQ 邮箱反馈；请尽量注明版本、页码与问题描述，便于核对和修订。

目录

1 为什么要学习编程 6

1.1 学习编程的热潮 6

1.2 为什么要学编程 6

1.3 什么是“编程” 7

1.4 怎么学编程 8

1.5 本章小结 8

2 基本数据类型 9

2.1 常量与变量 9

2.2 简单的屏幕输出 11

2.3 数据类型 11

2.4 如何计算变量或数据类型所占内存空间的大小 13

2.5 变量的赋值和赋值运算符 13

2.6 本章扩充内容 14

2.7 本章知识点小结 15

2.8 本章常见错误小结 16

3 基本算术运算 17

3.1 C 运算符和表达式 17

3.2 宏常量与宏替换 19

3.3 const 常量 20

3.4 自动类型转换与强制类型转换运算符 20

3.5 常用的标准数学函数 21

3.6 本章知识点小结 22

3.7 本章常见错误小结 22

4 键盘输入和屏幕输出 24

4.1 单个字符的输入/输出 24

4.2 数据的格式化屏幕输出 25

4.3 数据的格式化键盘输入 26

4.4 本章扩充内容 27

4.5 本章知识点小结 28

4.6 本章常见错误小结 28

5 选择控制结构 29

5.1 生活中与计算机中的问题求解方法 29

5.2 算法的概念及其描述方法 29

5.3 关系运算符与关系表达式 30

5.4 用于单分支控制的条件语句 31

5.5 用于双分支控制的条件语句 31

5.6 条件运算符和条件表达式 31

5.7 用于多分支控制的条件语句 32

5.8 用于多路选择的 switch 语句 33

5.9 逻辑运算符和逻辑表达式 34

5.10 本章扩充内容 34

5.11 本章知识点小结 36

5.12 本章常见错误小结 36

6 循环控制结构 37

6.1 循环控制结构与循环语句 37

6.2 计数控制的循环 39

6.3 嵌套循环 40

6.4 条件控制的循环 40

6.5 流程的转移控制 41

6.6 本章扩充内容 42

6.7 本章知识点小结 43

6.8 本章常见错误小结 44

7 函数与模块化程序设计 45

7.1 分而治之与信息隐藏 45

7.2 函数的定义 45

7.3 向函数传递值和从函数返回值 46

7.4 函数的递归调用和递归函数 48

7.5 变量的作用域和生存期 49

7.6 模块化程序设计 50

7.7 本章扩充内容——代码风格 52

7.8 本章知识点小结 53

7.9 本章常见错误小结 53

8 数组和算法基础 54

8.1 一维数组的定义和初始化 54

8.2 二维数组的定义和初始化 55

8.3 向函数传递一维数组 57

8.4 排序和查找 58

8.5 向函数传递二维数组 60

8.6 本章知识点小结 61

8.7 本章常见错误小结 62

9 指针 63

9.1 变量的内存地址 63

9.2 指针变量的定义和初始化 64

9.3 间接寻址运算符 65

9.4 按值调用与模拟按引用调用 66

9.5 用指针变量作函数参数的程序实例 67

9.6 函数指针及其应用 68

9.7 本章知识点小结 70

9.8 本章常见错误小结 70

10 字符串 71

10.1 字符串字面量 71

10.2 字符串的存储 71

10.3 字符指针 72

10.4 字符串的访问和输入/输出 73

10.5 字符串处理函数 74

10.6 向函数传递字符串 75

10.7 从函数返回字符串指针 76

10.8 本章扩充内容 76

10.9 本章知识点小结 78

10.10本章常见错误小结 78

11 指针和数组 79

11.1 指针和一维数组间的关系 79

11.2 指针和二维数组间的关系 80

11.3 指针数组及其应用 81

11.4 动态数组 82

11.5 本章扩充内容 84

11.6 本章知识点小结 85

11.7 本章常见错误小结 86

12 结构体和数据结构基础 87

12.1 从基本数据类型到抽象数据类型 87

12.2 结构体的定义 87

12.3 结构体数组的定义和初始化 90

12.4 结构体指针的定义和初始化 91

12.5 向函数传递结构体 91

12.6 共用体 92

12.7 枚举数据类型 93

12.8 动态数据结构——单向链表 93

12.9 本章扩充内容 95
12.10本章知识点小结 96
12.11第十一节：本章常见错误小结 97
13 文件操作 98
13.1 二进制文件和文本文件 98
13.2 文件的打开和关闭 98
13.3 按字符读写文件 99
13.4 按格式读写文件 100
13.5 按数据块读写文件 101
13.6 本章扩充内容 102
13.7 本章知识点小结 103
13.8 本章常见错误小结 103
14 简单的游戏设计 105
14.1 动画的基本原理 105
14.2 迷宫游戏 106
14.3 Flappy bird 游戏 107
14.4 本章知识点小结 109
14.5 本章常见错误小结 109

第 1 章

为什么要学习编程

1.1 学习编程的热潮

1.1.1 席卷全球的“编程一小时”

2013 年，公益网站 code.org 发起了每年 12 月持续一周的活动 Hour of Code，中文译作“编程一小时”。它的定位是“介绍计算机科学一小时的活动，旨在揭秘编程并说明它是任何人都可以学习的基础知识”。这项活动得到时任美国总统奥巴马以及扎克伯格、比尔·盖茨等科技界人士的支持，迅速从美国扩散到全球。

这件事说明的并不是“人人要当程序员”，而是：编程已经由专业技能变成了信息世界的通用工具，和识字、算术一样属于基础素养。

1.1.2 资本汹涌的少儿编程

2017–2018 年，资本大量进入少儿编程赛道，逻辑并不是“程序员缺口大”，而是押注“未来受教育人口普遍要接受编程教育”。资本判断的是趋势，不是课程内容本身——看懂这一点，就不必被“风口”牵着走。

1.1.3 “再不学编程就晚了”

这种恐慌来自 2016 年 AlphaGo 以 4:1 战胜李世石。但需要注意的是：AlphaGo 靠的是**高速搜索加海量数据存储**，并不是像人一样“思考”。它写不出新的算法思想，只能在你给定的规则空间里搜索。所以“学会编程就能凌驾于人工智能之上”并不成立，真正稀缺的是**提出问题和设计求解方法**的能力。

> **核心考点**　把“学编程”理解成“学一门手艺”会越学越窄；理解成“学一种描述计算过程的语言”才会越学越宽。前者随工具淘汰，后者可以迁移到任何语言。

1.2 为什么要学编程

“人人学编程”不等于“人人都是程序员”。程序员不是人人都想当，也不是人人都能做。学编程的普遍意义在于获得**计算思维**（Computational Thinking）。

这一概念由美国卡内基·梅隆大学计算机科学系前系主任周以真（Jeannette Wing）于 2006 年在《Computational Thinking》一文中提出。她的核心论断是：计算思维代表着一种普遍的认识和一类普适的技能，每一个人——不仅仅是计算机科学家——都应热心于它的学习和运用。

把计算思维落到程序设计上，可以拆成四个可操作的要素：

表 1.1: 计算思维的四个要素及其在 C 程序设计中的落点

要素	含义	在 C 语言中的落点
抽象	丢掉与求解无关的细节，只保留决定结果的特征	用数据类型刻画出“一个整数”“一个字符”；用函数接口隐藏实现
递归	把大问题化为同型的更小问题	递归函数必须有终止条件；栈帧的压入与弹出
复用	一次写对，多处调用	库函数、自定义函数、头文件、宏
折中	在正确、效率、可读、可移植之间取舍	用 double 保精度还是用 int 求速度；空间换时间

教材给出的两条主线同样值得记住：一是**程序设计方法由自底向上走向自顶向下**，二是**数据结构由简单走向复杂**。前七章沿着“简单数据”这条线展开，第 8 章之后才进入数组、指针、字符串、结构体等复杂数据。

1.3 什么是“编程”

编程的实质是：把人对问题的求解思路，转写成计算机能够逐步执行、且每一步都无歧义的指令序列。这个转写过程包含三个层次：

- **形式层**（语法）：写出来的东西必须符合语言的词法与语法规则，否则编译器拒绝翻译；
- **语义层**（含义）：每条语句在机器上真正做了什么——变量在内存的哪一格、值怎么变；
- **方法层**（思想）：把问题分解成什么样的步骤，用什么算法，数据结构怎么组织。

初学者最常见的困境是只盯着形式层：语法背得很熟，但拿到题不知道第一步写什么。原因就在于方法层没有建立——而这恰恰是这门课真正要训练的东西。

> **核心考点** 程序不是“写给人看的作文”，也不是“写给机器看的咒语”。它是**人与机器之间的合同**：人负责把方法想清楚，机器负责把每一步算到底。合同里任何一句有歧义，机器都会按它自己的理解执行，而且不会提醒你。

1.4 怎么学编程

1.4.1 一切都是计算

计算机能做的事只有一件：按指令对内存里的数据进行极为简单的操作（读、写、加、比、跳），然后以极高的速度和极大的数量把简单操作堆成复杂结果。看清这一点，就不会再问“为什么计算机这么笨，连这个都不会”，而会问“这件事能不能拆成它会做的那些简单操作”。

1.4.2 学编程不是学语法

语法是必需的，但它只是入场券。教材的写法是从实际问题出发、在案例的不断深化中引出知识点；同一段程序会用四种写法改写（for / while / do-while / goto），目的不是让你背四种语法，而是让你看清**同一个算法思想可以用不同控制结构表达**。所以判断自己学会没有，不看“能不能背出语法格式”，而看“给定一个新问题，能不能拆出步骤并选对结构”。

1.4.3 动手，动手，再动手

1. 每个知识点至少亲手敲一遍、编译一遍、跑一遍，不要只看。
2. 故意制造错误：删掉初始化、写错分号、漏掉取地址符，看编译器报什么、运行结果怎么错。教材每章都有“常见错误小结”，正是为这一步准备的。
3. 改需求：把“求 1 到 n 之和”改成“求奇数之和”“求平方和”，检验自己是否真的掌握了模板而不只是记住了答案。

1.5 本章小结

本章不含 C 语法，但它决定了后面 13 章的学习方式。三个结论：

- 学编程的目的是获得计算思维（抽象、递归、复用、折中），不是背语法；
- 编程是把求解方法转写成无歧义指令序列的过程，难点在方法层；
- 唯一有效的路径是动手：敲、错、改、再改需求。

从第 2 章开始进入具体语法。届时请始终带着一个问题读代码：**这条语句在内存里到底改动了什么？**

第 2 章

基本数据类型

2.1 常量与变量

C 语言程序处理的数据有**常量**和**变量**两种形式。常量在程序运行过程中值不变，变量则对应内存中一个可读写的存储单元。

2.1.1 常量

常量按其表示的数据种类分为四类：整型常量、实型常量、字符型常量、字符串字面量。

整型常量

整型常量可以用四种进制书写，区别在于前缀：

表 2.1: 整型常量的进制表示

进制	前缀	示例	十进制值
十进制	无	`123`、`-45`	123、−45
八进制	数字 `0`	`017`	15
十六进制	`0x` 或 `0X`	`0x1F`	31
二进制	无（C99 起可用 `0b`，教材未采用）	——	——

书写整型常量时要注意后缀表示类型：`123L` 是长整型，`123U` 是无符号整型，`123UL` 是无符号长整型。八进制常量以 `0` 开头这一点极易出错：`017` 不是十七而是十五，`0` 后面若误写了 `8` 或 `9`（如 `018`）则是非法常量。

实型常量

实型常量有**小数形式**和**指数形式**两种写法：

- 小数形式：必须带小数点，如 `3.14`、`.5`、`5.`、`-0.001`；
- 指数形式：由尾数、字母 `E`（或 `e`）和指数三部分组成，如 `1.23456e4` 表示 1.23456×10^4，`2.5E-3` 表示 2.5×10^{-3}。

指数形式有两条硬规定：`E` 前后的**尾数和小数部分都不能省略**，即 `e4`、`1.2e` 都是非法的；指数必须是整数，`1.2e1.5` 也是非法的。实型常量默认为 `double` 型，加后缀 `f`（如 `3.14f`）则为 `float` 型。

字符常量与字符串字面量

字符常量是用一对**单引号**括起来的单个字符，如 `'a'`、`'A'`、`'5'`。字符在内存中存的是它的 ASCII 码值，因此 `'a'` 与整数 97 在数值上等价，可以参与算术运算。

反斜杠开头的**转义字符**用来表示无法直接写出的控制字符或特殊符号：

表 2.2: 常用转义字符

转义字符	含义	说明
`\n`	换行	光标移到下一行行首
`\t`	水平制表	跳到下一个制表位
`\r`	回车	光标回到本行行首
`\b`	退格	光标左移一格
`\\`	反斜杠	输出一个 `\`
`\'` / `\"`	单/双引号	在字符串中输出引号本身
`\0` 或 `\000`	空字符	ASCII 值为 0，字符串结束标志

字符串字面量（常量）是用一对**双引号**括起来的若干字符，如 `"Hello"`。两者的关键差别：`'a'` 是字符常量，占 1 个字节；`"a"` 是字符串，占 2 个字节（字符 `a` 加一个 `'\0'` 结束标志）。所以 `'a'` 与 `"a"` 不能混用，单引号只能括一个字符，双引号不能用来表示字符。

2.1.2 变量

变量在使用之前**必须先定义**，定义变量的一般形式为

类型关键字　变量名;

变量名是用户定义的标识符，用于标识内存中一个具体的存储单元。命名必须遵守：

1. 第一个字符必须是字母（A–Z、a–z）或下划线 `_`；
2. 后续字符可以是字母、数字或下划线；
3. 不能包含空格或其他特殊字符；
4. 不能使用 C 的关键字（如 `int`、`return`、`if`）；
5. **区分大小写**：`newValue` 与 `newvalue` 是两个不同的变量。

C 语言允许定义变量的同时**初始化**（赋初值）；一条语句可同时定义多个同类型变量，之间用逗号分隔。但要注意下面两种写法的区别：

```
int a = 0, b = 0, c = 0;    /* 正确：三个变量各自初始化 */
int a = b = c = 0;          /* 错误：b、c 此时尚未定义 */
```

还有一个易被忽视的约束：**变量必须在可执行语句之前定义**。把 `int n;` 写在 `printf` 之后，C89 编译器会报错（C99 之后允许，但为了可移植性，仍应统一写在函数开头）。

> **核心考点** 定义变量时问两个问题：它要存什么类型的数据（决定类型关键字）、它的值从哪里来（决定是否需要初始化）。未初始化的局部变量里是**残留的垃圾值**，不是 0——这是“结果莫名其妙”的最常见来源。

2.2 简单的屏幕输出

一个能运行的最小 C 程序结构如下：

```c
#include <stdio.h>
int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

第一行以 `#` 开头且不以分号结尾，它不是 C 语句，而是**编译预处理命令**。尖括号内的是**头文件**，`stdio.h` 中声明了标准输入/输出函数的原型；不包含它而直接调用 `printf()`，编译器会警告甚至报错。

关于 `printf()` 需要先建立三点认识：

- 它的作用是**按格式**把数据输出到屏幕，最简单的用法是输出一个字符串；
- 用一对双引号括起来的若干字符称为字符串，字符串中可以包含转义字符；
- `main` 函数的返回值 `0` 表示程序正常结束，这个 0 会返回给操作系统。

2.3 数据类型

C 语言把数据类型分为基本类型、构造类型、指针类型和空类型四大类。之所以要区分类型，是因为不同类型的数据**取值范围不同、表示形式不同、可参与的运算也不同**。

几点补充：定义整型变量时若不指定 `unsigned`，隐含就是**有符号型**，而 `signed` 通常省略不写；`char` 型在算术运算中按整型处理（存放字符的 ASCII 码）；`void` 表示“无类型”，常用作无返回值的函数返回类型或无类型的指针。

表 2.3: C 语言的数据类型分类

分类	关键字	变量声明实例	详述章节
基本类型	基本整型 `int`	`int a;`	第 2 章
	长整型 `long int`	`long int a;` 或 `long a;`	第 4 章
	长长整型 `long long`	`long long int a;` 或 `long long a;`	第 4 章
	短整型 `short int`	`short int a;` 或 `short a;`	第 4 章
	无符号整型 `unsigned`	`unsigned int a;` 等	第 4 章
	单精度实型 `float`	`float a;`	第 4 章
	双精度实型 `double`	`double a;`	第 4 章
	长双精度实型 `long double`	`long double a;`	第 4 章
	字符型 `char`	`char a;`	第 4 章、第 10 章
	枚举类型 `enum`	`enum response{no, yes};`	第 12 章
构造类型	数组	`int score[10];`	第 8 章
	结构体 `struct`	`struct date{int y; int m; int d;};`	第 12 章
	共用体 `union`	`union{int i; char c;} u;`	第 12 章
指针类型	——	`int *ptr;  char *pStr;`	第 9–11 章
空类型	`void`	`void Sort(int a[], int n);`	第 8 章、第 11 章

2.4 如何计算变量或数据类型所占内存空间的大小

内存空间的基本单位是**字节**（byte），1 字节 = 8 **位**（bit）。位是存储信息的最小单位，字节是编址的基本单位。

`sizeof` 是 C 语言的**关键字**，不是函数名；`sizeof()` 是专门用于计算指定数据类型字节数的**运算符**。它有两个用法：

```
sizeof(int)       /* 计算 int 型数据所占的字节数 */
sizeof(a)         /* 计算变量 a 所占的字节数 */
```

`sizeof` 在**编译时**求值，不增加程序运行时间开销，但能显著增强程序的可移植性——它是“绝不对变量所占字节数想当然”这一规则的工具保障。

表 2.4: 常见数据类型所占字节数（典型取值，以 `sizeof` 实测为准）

类型	32 位环境字节数	64 位环境字节数	说明
`char`	1	1	恒为 1 字节，C 标准规定
`short`	2	2	长度不长于 `int`
`int`	4	4	通常与机器字长相同
`long`	4	4（Windows）/8（Linux）	长度不短于 `int`
`long long`	8	8	C99 起
`float`	4	4	约 7 位有效数字
`double`	8	8	约 15–16 位有效数字
`long double`	12 或 16	16	因编译器而异
指针	4	8	与地址总线宽度一致

C 标准并没有规定各种整型数据在内存中占多少字节，只要求**长整型的长度不短于基本整型，短整型的长度不长于基本整型**。同一类型在不同编译器和不同计算机系统中所占字节数可能不同，因此跨平台程序必须用 `sizeof` 而不是硬编码的常数。

2.5 变量的赋值和赋值运算符

赋值运算符 `=` 用于给变量赋值，由赋值运算符及其两侧操作数组成的表达式称为**赋值表达式**。在赋值表达式后加分号即构成**表达式语句**：

```
x = 1;          /* 赋值表达式 + 分号 = 赋值语句 */
```

赋值运算有严格的**方向性**：把右侧表达式（右值）的值赋给左侧的变量，并**不表示**“等号两侧的值相等”。因此等号左侧只能是标识一个特定存储单元的变量名，`1 = x`、`x + 1 = y` 都是非法的。

几条与运算规则相关的结论：

- **优先级**：算术运算符的优先级高于赋值运算符，所以 `x = a + b` 先算加法再赋值；
- **赋值表达式的值**：被规定为运算完成后左操作数的值，类型与左操作数相同。因此 `x = x + 1` 的值就是 `x` 加 1 之后的值；
- **结合性**：赋值运算符是**右结合**的（自右向左），而需要两个操作数的算术运算符是左结合的。

右结合性直接解释了多重赋值的执行顺序：

```
a = b = c = 0;          /* 等价于 a = (b = (c = 0)); */
```

执行时从右向左，把最右侧表达式的值依次赋给左侧的变量，常用于给多个变量赋相同初值。而 `x / y * z` 因为是左结合，等价于 `(x / y) * z` 而非 `x / (y * z)`——两种结合性混在一起是初学阶段最典型的错误来源。

2.6 本章扩充内容

2.6.1 有符号整数和无符号整数

两者的区别在于**怎样解释整数的最高位**。对无符号整数，最高位被解释为数据位；对有符号整数，最高位被解释为**符号位**：0 表示正数，1 表示负数。

由于符号位占去 1 位，同样字节数的有符号整数所能表示的最大值只有无符号整数的一半。以 2 字节（16 位）为例：

表 2.5: 2 字节整型数的取值范围

类型	取值范围	说明
有符号 `short`	$-32768 \sim 32767$	最小值 -2^{15}，最大值 $2^{15}-1$
无符号 `unsigned short`	$0 \sim 65535$	最大值 $2^{16}-1$

负数在计算机中以**二进制补码**（complement）形式存储。由原码求补码的规则是：**符号位不变**，其余各位取反得反码，再在末位加 1。例如 -1 的原码为 `1000 0000 0000 0001`，反码为 `1111 1111 1111 1110`，加 1 后得补码 `1111 1111 1111 1111`（全 1）。正数的原码、反码、补码三者相同。

用补码表示有两条好处：

1. **减法可化为加法**。计算 $7-6$ 时，把 $+7$ 的补码与 -6 的补码相加，舍去最高位的进位，结果正是 $+1$，因此硬件只需一个加法器；
2. **0 的表示唯一**。若用原码，$+0$ 与 -0 的编码不同；而按补码计算，-0 的反码加 1 后进位被舍去，又变回全 0，与 $+0$ 一致。

> **核心考点** 全 1 的 16 位编码，按有符号数解释是 −1，按无符号数解释是 65535。**同一串二进制位，类型不同，含义就不同**——这正是数据类型存在的意义。把无符号数当有符号数用（或反之），是“输出莫名其妙的大数/负数”的根本原因。

2.6.2 实型数据在内存中的存储格式

实数无论写成小数形式还是指数形式，在计算机内部都按**浮点形式**存储。浮点形式是相对于定点形式而言的：定点表示中小数点位置固定（整数是小数点固定在数值位之后的特例，表示范围很小、很死板），而浮点数的小数点位置**可以浮动**，因此能表示很大或很小的数。

浮点数把实数分成**阶码**和**尾数**两部分：

$$N = S \times r^J$$

其中 S 为尾数（正负均可，一般规定为纯小数形式），J 为阶码（正负均可，但必须是整数），r 为基数。对二进制数而言 $r = 2$，即 $N = S \times 2^J$。存储时依次存放阶码符号、阶码数值、尾数符号、尾数数值。

两条必须记住的结论：

- **阶码所占的位数决定实数的表数范围**（能表示多大、多小的数）；
- **尾数所占的位数决定实数的精度**（有效数字有几位），尾数的符号决定实数的正负。

用更多的位存尾数可以提高精度但缩小表数范围；用更多的位存阶码可以扩大表数范围但降低精度。因此**浮点数并非真正意义上的实数，只是其在某种范围内的近似值**。C 标准没有明确规定三种浮点类型的阶码和尾数各占多少位，不同编译器分配不同。

2.6.3 单精度实型和双精度实型的有效位数

把同一个实型常量分别赋给 `float` 与 `double` 变量并输出，末尾几位数字会不同。原因在于二者能接收的**有效数字位数**不同：`double` 型变量一般可接收 15–16 位有效数字，而 `float` 型变量仅能接收约 7 位有效数字，有效数字之后输出的数字都是不准确的。

由此得到一条实践规则：**把 `double` 数据赋给 `float` 变量时可能发生数据截断，产生舍入误差**。需要较高精度（如累加求和、迭代运算）时应使用 `double`；只有确实要节省空间或与特定接口对接时才用 `float`。

2.7 本章知识点小结

- 数据分常量与变量；常量有整型、实型、字符型、字符串字面量四类；变量必须先定义后使用，区分大小写；
- 整型常量四种进制靠前缀区分（`0` 八进制、`0x` 十六进制），实型常量有小数与指数两种写法；
- 字符常量用单引号、字符串用双引号，字符串末尾隐含 `'\0'`；

- 数据类型分基本、构造、指针、空类型；类型决定取值范围、存储形式与可参与的运算；
- `sizeof` 是编译时求值的运算符，用于计算类型或变量的字节数，是可移植性的保障；
- 赋值运算符具有方向性和右结合性，赋值表达式的值等于运算后左操作数的值；
- 内存模型：负数用补码存储，阶码决定表数范围，尾数决定精度，浮点数是近似值。

2.8 本章常见错误小结

表 2.6: 第 2 章常见错误与纠正

易错写法	错在哪里	纠正
`int newVa1ue;` 后写 `newvalue = 0;`	变量名区分大小写，等于未定义就使用	名字前后一致
`printf` 之后写 `int n;`	变量必须在可执行语句之前定义	定义集中写在函数开头
`int a = b = c = 0;`	`b`、`c` 此时尚未定义	`int a=0,b=0,c=0;`
`float x = 3.5;`	常量类型为 `double`，赋值时发生转换	用 `3.5f` 或改变量为 `double`
`'a'` 与 `"a"` 混用	单引号是字符，双引号是字符串	字符用单引号
`017` 当作十七	前导 0 表示八进制，实为 15	需要十进制就不要写前导 0
变量未初始化直接参与运算	局部变量中是残留垃圾值，不是 0	定义时显式初始化

第 3 章

基本算术运算

3.1 C 运算符和表达式

C 语言共提供 34 种运算符。按所需操作数（Operand，也称运算对象）的个数，运算符分为一元（单目）、二元（双目）和三元（三目）三种；条件运算符是 C 语言中唯一的三元运算符。

3.1.1 算术运算符和表达式

由算术运算符及其操作数组成的表达式称为算术表达式，操作数既可以是常量、变量，也可以是函数调用。C 的算术运算符共 5 个，其优先级与结合性见表 3.1。

表 3.1: 算术运算符的优先级与结合性

运算符	名称	操作数个数	优先级	示例与结果
-（一元）	取相反数	1	最高	-5 ⇒ -5
*	乘法	2	较高	3*4 ⇒ 12
/	除法	2	较高	12/5 ⇒ 2；12/5.0 ⇒ 2.4
%	求余	2	较高	11%5 ⇒ 1
+	加法	2	最低	5+1 ⇒ 6
-（二元）	减法	2	最低	5-1 ⇒ 4

（1）运算结果与操作数类型有关。这是 C 的算术运算区别于数学运算的最重要一点。两个整数相除的商仍为整数，小数部分被直接舍去（不是四舍五入），称为**整数除法**（Integer Division）；只要有一个操作数是实数，就按**浮点数除法**（Floating Division）进行，整数操作数在运算前被自动转换为实数。

```
int   a = 12, b = 5;
double c = 12.0;
a / b      /* 2      : 整数除法，舍去小数部分 */
a / 5.0    /* 2.4    : 浮点数除法，a 先转成 double */
c / b      /* 2.4    : 浮点数除法 */
(float)a / b  /* 2.4  : 先把 a 强转为 float 再相除 */
```

（2）求余运算限定两个操作数都是整型，不能对实数求余。求余时左操作数为被除数、右操作数为除数，结果是整除后的余数（Remainder），**余数的符号与被除数的符号相同**：

$$11\%5 = 1, \qquad (-11)\%5 = -1, \qquad 11\%(-5) = 1.$$

（3）C 语言没有幂运算符。数学上的 x^y 必须写成 `pow(x, y)`（需 `<math.h>`），而 $\sqrt{x}$ 写成 `sqrt(x)`。

（4）优先级与结合性。取相反数最高，其次是 `* / %`，最低是 `+ -`；`* / %` 三者同级，`+ -` 二者同级。同级混合运算按结合性决定顺序：一元取相反数为右结合（自右向左），其余算术运算符为左结合（自左向右）。任何表达式中都优先计算圆括号内的值，因此当优先级容易出错时，宁可用括号把意图写死。

一个经典的例子是把三位整数的各位数字拆出来：`153 % 10` 得个位 3，`153 / 100` 得百位 1，而十位既可用 `(153 / 10) % 10` 得到，也可用 `(153 - 1*100) / 10` 得到。注意后一种写法里的圆括号不能省，因为 `/` 的优先级高于 `-`。

3.1.2 复合的赋值运算符

涉及算术运算的复合赋值运算符有 5 个：`+=`、`-=`、`*=`、`/=`、`%=`。运算符的两个字符之间不能有空格。

它的形式与等价关系是：

```
a op= b   ⟺   a = a op (b)
```

即：先把等号右侧的表达式整体用圆括号括起来，再把等号左侧的运算符移到等号右侧，最后在等号左侧补上被赋值的变量名。

表 3.2: 复合赋值运算及其等价形式

复合赋值	等价形式	说明
`n += m + 1`	`n = n + (m + 1)`	右侧整体加括号
`n -= m + 1`	`n = n - (m + 1)`	同上
`n *= m + 1`	`n = n * (m + 1)`	**不等价**于 `n = n * m + 1`
`n /= m + 1`	`n = n / (m + 1)`	整数除法时结果可能为 0
`n %= m + 1`	`n = n % (m + 1)`	两侧必须都是整型

复合赋值书写更简洁，生成的代码执行效率也略高（只需对左操作数求值一次）。但要记住它**不是数学等式**，不能与 `==` 混淆。

3.1.3 增 1 运算符和减 1 运算符

`++` 与 `--` 都是一元运算符，作用是把变量的值加 1 或减 1，有前缀与后缀两种形式：

- **前缀形式** `++i`：先对 `i` 加 1，再使用 `i` 的新值；

- **后缀形式** `i++`：先使用 `i` 的当前值，再对 `i` 加 1。

作为独立语句使用时，`i++;` 与 `++i;` 完全等价，都是把 `i` 加 1。差别只出现在**把增 1 表达式当作值来用**的场合：

```
int i = 1, j;
j = i++;    /* j = 1, i = 2 : 先把旧值 1 赋给 j, 再自加 */
j = ++i;    /* i 先变 2, j = 2 */
```

使用时有几条硬规则：

1. 增 1、减 1 的操作数必须是**变量**（可修改的左值），`5++`、`(i+j)++` 都是错误的；
2. 不要对一个算术表达式或常量使用 `++`、`--`；
3. 不在同一条语句中对同一变量多次使用 `++`、`--`。C 标准没有规定这类表达式的求值顺序，编译器可以自由重排，不同编译器（甚至同一编译器的不同优化级别）可能给出不同结果。良好的程序设计风格不写这种晦涩的表达式。

> **核心考点** 最容易失分的一类题是“`i++ + ++i` 等于多少”。正确的回答不是某个数，而是“这是一个未定义行为，不该写”。考场上遇到这类题，先判断它是否属于同一变量的多次修改，再决定能不能算。

3.2 宏常量与宏替换

程序中直接写死的常数称为**幻数**（Magic Number），例如圆周率写成 `3.14159`。幻数带来三类麻烦：可读性差（过一段时间连作者都想不起这个常数是什么）、多处书写容易写错、需要修改时必须逐处查找且容易遗漏。

解决办法之一是用**宏定义**给它起个名字。宏常量（Macro Constant，也称符号常量）是用一个标识符表示的常量，由编译预处理指令 `#define` 定义：

```
#define PI 3.14159      /* 宏名习惯全部大写；宏名与字符串之间无等号 */
#define MAX 100         /* 行末不加分号：宏定义不是 C 语句 */
```

预处理阶段会把源程序中该宏名**原样替换**成后面的字符串，这个过程称为**宏替换**（Macro Substitution）。替换是“傻瓜式”的纯文本操作，因此有几条必须记住的规则：

1. 宏名与替换字符串之间可以有多个空白符，但**不能写等号**。写成 `#define PI = 3.14159` 后，`2*PI*r` 会展开成 `2*=3.14159*r`，产生语法错误；
2. 行末**不加分号**。加了分号，分号会一起被替换进去；
3. 宏替换**不做语法检查**，也不做类型检查；只有宏展开后的源程序在编译时才会暴露语法错误；
4. 替换字符串里的**双引号内**与宏名相同的字符**不会被替换**；

5. 宏名没有数据类型、没有作用域限制，从定义处到文件末尾一直有效。

3.3 const 常量

宏常量的最大问题是**没有数据类型**、编译器不做类型检查。若要一个“有类型的常量”，使用 const 类型修饰符：

```
const double PI = 3.14159;   /* 有类型、只读 */
const int    MAX = 100;
```

把 const 放在类型名之前，即可把其后的标识符声明为具有该类型的 const 常量。编译器把 const 常量放在**只读存储区**（只读数据段），程序运行中不允许修改它的值，因此 const 常量**只能在定义时赋初值**。

表 3.3: 宏常量与 const 常量的对比

比较项	宏常量 #define	const 常量
处理阶段	预处理阶段做文本替换	编译阶段处理，运行时有存储单元
数据类型	无	有
类型检查	不做	做，能发现类型错误
作用域	无（定义处到文件末尾）	块级作用域
调试	符号表中无该名字	符号表中有，可观察其值
修改	修改定义处即可（但无保护）	不允许修改，误改会编译报错

常规建议是优先用 const：它有类型、能检查、能调试。仅在需要“纯文本替换”（如条件编译、宏函数）时才用 #define。

3.4 自动类型转换与强制类型转换运算符

3.4.1 表达式中的自动类型转换

相同类型的操作数运算，结果类型与操作数类型相同；不同类型混合运算时，编译器先把所有操作数转换成**取值范围较大**的类型，称为**类型提升**（Type Promotion）。转换方向由低到高：

$$\texttt{char, short} \rightarrow \texttt{int} \rightarrow \texttt{unsigned int} \rightarrow \texttt{long} \rightarrow \texttt{unsigned long} \rightarrow \texttt{long double}$$

（float 在运算中一律提升为 double；C99 中还有 long long、unsigned long long，位于 long、unsigned long 与浮点类型之间。）

两条要点：

- 所有 `char` 与 `short` 都先提升为 `int`，这一步称为**整数提升**（Integral Promotion）；
- 一个特例：若一个操作数是 `long`、另一个是 `unsigned`，且 `unsigned` 的值不能用 `long` 表示，则两者都转换为 `unsigned long`。

转换之后每对操作数类型相同，所以结果类型自然与操作数类型相同。正因为存在这种规则，`1/2` 得 0 而 `1.0/2` 得 0.5——参与运算的类型决定了运算本身，而不是你想让结果是什么。

3.4.2 赋值中的自动类型转换

赋值时若左侧变量的类型与右侧表达式的类型不同，右侧的值会被自动转换成**左侧变量的类型**再存入。这是一把双刃剑：

- 由取值范围小的类型转向大的类型是**安全**的；
- 由取值范围大的类型转向小的类型**不安全**，可能发生数据信息丢失或数值溢出（例如把 `pow()` 的结果赋给 `long`，小数位丢失，结果超出范围时还会溢出）；
- 好的编译器会对这类隐式转换发出警告。

3.4.3 强制类型转换运算符

强制类型转换（casting）运算符的写法是

(类型) 表达式

它是一个一元运算符，与其他一元运算符优先级相同。与自动转换的“隐式”相对，强转是**显式**声明“我确实要这样转”，能消除隐式转换带来的隐患。

```
int m = 5;
m / 2                /* 2        : 整数除法，小数位被舍去 */
(float)(m / 2)       /* 2.000000 : 先整除得 2，再在 2 后面补小数位 */
(float)m / 2         /* 2.500000 : 先把 m 转成实数，再做浮点除法 */
m                    /* 5        : 强转不改变 m 本身的类型和值 */
```

最后一行是关键：`(float)m` 只是取 `m` 的值临时转换成一个实数**值**参与运算，`m` 的类型与内容都没有被改变。

3.5 常用的标准数学函数

C 标准数学函数库提供了丰富的数学函数，常用的有 `sqrt`（平方根）、`fabs`（绝对值）、`pow`（幂）、`exp`、`log`、`log10`、`sin`、`cos`、`tan` 等，完整列表见教材附录。使用时必须在程序开头包含头文件：

```
#include <math.h>
double s = sqrt(2.0);        /* 1.414214 */
double t = fabs(-3.5);       /* 3.5 : 实型取绝对值，整型用 abs() */
```

```
double u = pow(2.0, 10.0);   /* 1024.0 : C 没有幂运算符 */
double v = sin(3.14159 / 6); /* 0.5    : 三角函数参数是弧度，不是角度 */
```

把数学公式写成 C 表达式时，最典型的三类错误是：

1. 漏写乘号。数学上 $s(s-a)(s-b)$ 可以省略乘号，C 语言中 `*` 一个都不能省；
2. 整数除法。把 $\frac{1}{2}(a+b+c)$ 写成 `1/2*(a+b+c)` 得 0，应写 `1.0/2*(a+b+c)` 或 `(a+b+c)/2.0`；
3. 强转位置错。`(float)((a+b+c)/2)` 是先整除再强转，小数位已经丢了；正确写法是把分子先转成实数，即 `(float)(a+b+c)/2`。

> **核心考点** 判断一个算术表达式对不对，不看它“看起来像不像数学公式”，而看**每个中间结果的类型**。养成逐层标注类型的习惯：每一步是整数还是实数、会不会溢出、会不会被截断。

3.6 本章知识点小结

- 算术运算的结果类型由操作数类型决定：整数除法舍小数，求余只能用于整型，余数符号随被除数；
- 优先级：一元取反 > `* / %` > `+ -`；结合性：一元右结合，其余左结合；括号优先；
- 复合赋值 `a op= b` 等价于 `a = a op (b)`，右侧整体加括号；
- `i++` 与 `++i` 只在“取表达式的值”时有差别，同一变量不要在同一条语句里多次自增；
- 常量两种写法：宏常量（无类型、纯替换）与 const 常量（有类型、只读、只能定义时初始化）；
- 混合运算按“由低到高”自动提升；赋值按“右侧转成左侧”转换；强转不改变变量本身。

3.7 本章常见错误小结

表 3.4: 第 3 章常见错误与纠正

易错写法	错在哪里	纠正
`1/2*x`	整数除法，结果为 0	`1.0/2*x`
`(float)(m/2)`	先整除再强转，精度已丢失	`(float)m/2`
`a%2.0`	求余的操作数必须为整型	改为整型操作数
`#define PI = 3.14`	多写了等号，宏展开成 `2*=3.14*r`	`#define PI 3.14`
`#define PI 3.14;`	多写了分号，分号进入表达式	去掉分号
`#define MAX 10` 后写 `max`	宏名区分大小写	大小写一致
`const int N; N=5;`	const 常量只能定义时初始化	`const int N = 5;`
`5++` / `(a+b)++`	自增的操作数必须是变量	改为对变量操作
`sin(30)`	三角函数参数为弧度，不是角度	`sin(30*PI/180)`

第4章

键盘输入和屏幕输出

4.1 单个字符的输入/输出

4.1.1 字符常量

字符常量是用一对单引号括起来的单个字符。C 语言中，字符在内存里存的是它的 **ASCII 码值**，因此字符与整数在数值上是等价的，可以直接参与整型运算：

```
char ch = 'A';
printf("%c %d\n", ch, ch);      /* A 65 */
ch = ch + 32;                    /* 'A' -> 'a', 65 + 32 = 97 */
printf("%c %d\n", ch, ch);      /* a 97 */
```

观察 ASCII 码表可以发现一条常用规律：**小写字母的 ASCII 码值比相应大写字母大 32**，即 `'a'` 与 `'A'`、`'b'` 与 `'B'` 都相差 32。据此可轻便地实现大小写转换；反过来，`ch - 'a' + 'A'` 可把小写转成大写。这类“字符就是整数”的性质在字符分类、字符串处理中会反复用到。

字符常量书写有三条限制：单引号内**只能有一个字符**（`'ab'` 非法）；转义字符算一个字符（`'\n'` 合法）；`'5'` 是字符（ASCII 值 53），而 5 是整数，两者不能混用。

4.1.2 字符的输入/输出

只处理单个字符时，标准库提供了一对专用函数：

```
int getchar(void);          /* 从键盘读入一个字符，返回其 ASCII 值 */
int putchar(int c);         /* 向屏幕输出一个字符 */
```

```
ch = getchar();      /* 读入一个字符，赋给 ch */
putchar(ch);         /* 输出该字符 */
putchar('\n');       /* 输出一个换行控制符 */
```

要点如下：

- `getchar()` **没有参数**，它的返回值是读到的字符的 ASCII 码，所以既可以赋给 `char` 变量，也可以赋给 `int` 变量；
- `putchar()` 的参数是一个字符（或它的 ASCII 码值），一次只能输出一个字符；

- `getchar()` 从**输入缓冲区**读取字符，而不是每次按键都立即返回。用户按回车后，整行字符（**包括回车符**）才一起进入缓冲区；之后每次调用 `getchar()` 都直接从缓冲区取字符，只有缓冲区被读空后才会再次等待按键。这条机制是 4.4 节一系列问题的根源；
- 用 `printf("%c", ch)` 与 `putchar(ch)` 输出一个字符的效果等价；而 `printf("\n")` 与 `putchar('\n')` 也等价，区别只在于前者写在双引号内（字符串），后者写在单引号内（单个字符）。

4.2 数据的格式化屏幕输出

`printf()` 的一般调用形式为

```
printf(格式控制字符串, 输出值参数表);
```

格式控制字符串是用双引号括起来的字符串，包含两类内容：**格式转换说明符**（以 `%` 开头，指定数据的输出格式）和**普通字符**（原样输出）。**输出值参数表**是需要输出的数据项列表，可以是变量或表达式，之间用逗号分隔，其类型应与格式转换说明符**一一对应**。

表 4.1: printf 常用格式转换说明符

格式符	对应类型	说明
`%d`	`int`	十进制整数；`%ld` 对应 `long`
`%c`	`char`	按字符输出；用 `%d` 则输出其 ASCII 码值
`%f`	`float/double`	小数形式，隐含输出 6 位小数
`%e`	实型	指数形式
`%g`	实型	自动选择较短形式，并去掉无意义的 0
`%s`	字符串	输出字符串
`%o` / `%x`	整型	八进制 / 十六进制
`%u`	无符号整型	十进制无符号

域宽与精度可以写在 `%` 与格式字符之间：`%7.2f` 表示输出数据占 **7 个字符宽度**、保留 **2 位小数**。注意**小数点本身也占 1 个字符位置**，所以 `33.30` 共 5 个字符，前面会补 2 个空格。若数据实际宽度超过指定域宽，则按实际宽度输出，域宽不起作用。若精度写成 `.0`（如 `%7.0f`），相当于输出实数的取整结果。

```
double c = 33.300854, a = 88.247263;
printf("circumference = %f, area = %f\n", c, a);
/* 隐含 6 位小数: 33.300854, 88.247263 */
printf("%7.2f, %7.2f\n", c, a);
/* 33.30,   88.25 —— 各占 7 列, 右对齐, 小数保留 2 位 */
printf("%-7.2f|\n", c);
/* 33.30  | —— 负号表示左对齐, 右侧补空格 */
```

还有几个实用修饰：- 表示左对齐（默认右对齐），+ 表示正数也显示正号，0 表示用 0 填充空位（如 %05d）。printf() 的**返回值**是实际输出的字符个数，一般不用，但在拼接输出长度时有用。

> **核心考点** printf() 最常见的两类错误：一是格式符与参数类型不匹配（用 %d 输出实型、用 %f 输出整型），输出结果是不可预期的乱值；二是格式符个数与参数个数不等，多出的参数被忽略、缺少的参数会读到栈上的垃圾值。写完一行 printf，从左到右数一遍 % 的个数与逗号后面参数的个数是否相等。

4.3 数据的格式化键盘输入

scanf() 的一般调用形式为

scanf(格式控制字符串，参数地址表);

格式控制字符串同样由格式转换说明符和分隔符两部分组成；**参数地址表**给出的不是变量本身，而是变量**的地址**，因此每个参数前通常要加取地址运算符 &。

```
int a, b;
double x;
scanf("%d%d", &a, &b);       /* 输入两个整数，用空白分隔 */
scanf("%lf", &x);             /* 读入 double 必须用 %lf */
```

表 4.2: scanf 常用格式转换说明符及其输入规则

格式符	对应类型	输入规则
%d	int	跳过前导空白，读到非数字字符为止
%f	float	读实数
%lf	double	double 变量必须用 %lf
%c	char	**不跳过空白**，连空格、回车都当有效字符读入
%s	字符数组	跳过前导空白，读到空白字符为止
%2d	int	限定域宽，自动按指定宽度截取输入数据
%*d	——	忽略符，读入该数据但不赋给任何变量

输入分隔规则是最容易踩坑的地方：

- 数值型（%d、%f）格式符之间用**空白字符分隔**时，输入数据之间可用空格、回车或制表符任一种分隔，不必与格式串严格一致；
- 若格式串中两个格式符之间写了**普通字符**（如逗号），则用户输入时必须**原样输入**该字符。scanf("%d,%d", &a, &b) 要求输入 12,34；

- 若格式串中写了提示文字（如 `"a=%d,b=%d"`），用户也必须把这些文字原样敲进去，这在实际使用中很不友好，通常改为先用 `printf` 提示、再只写格式符；
- **不要在格式串末尾加 `\n`**。`scanf` 中的 `\n` 不是“等待换行”，而是“跳过所有空白字符”，会导致程序一直等待输入；
- `scanf()` 的**返回值**是成功读入的数据个数，可用于判断输入是否合法（如 `if (scanf("%d", &n) != 1)`）。

> **核心考点** `scanf` 忘记写 `&` 是初学者最高频的致命错误。此时编译器把变量的**值**当作地址，数据被写到非法内存单元，真正的变量却没有被赋值，运行时可能直接崩溃（非法内存访问），也可能“看起来没报错但结果是垃圾值”。看到“输入了却没生效”，第一件事就是回去数 `&`。

4.4 本章扩充内容

4.4.1 用%c 输入字符时存在的问题

`%c` 的特殊之处在于它**不跳过任何空白字符**。结合 4.1 节讲的缓冲区机制，就会出现这样的现象：

```
int data1, data2;  char op;
printf("Please enter the expression data1+data2\n");
scanf("%d%c%d", &data1, &op, &data2);
printf("%d%c%d = %d\n", data1, op, data2, data1 + data2);
```

用户输入 `12 + 3` 时，期望 `op` 读到 `'+'`，实际却把空格读给了 `op`，于是 `data2` 读到的是 `'+'`（其 ASCII 值 43），输出变成 `12 3 = 3141` 之类的怪结果。

更常见的版本是连续读入不同类型的数据：

```
scanf("%d", &a);      /* 用户输入 12 后按回车 */
scanf("%c", &b);      /* b 读到的是回车符，而不是下一个有效字符 */
```

原因同上一节：输入 12 后按下的**回车符留在了缓冲区**，下一次 `%c` 调用直接从缓冲区取走它。这类错误的典型症状是“第二次输入被跳过了”。

4.4.2 %c 格式符存在问题的解决方法

有两种标准做法，第二种更简单、可读性也更好：

```
/* 方法 1：用 getchar() 把缓冲区中的回车符读掉 */
scanf("%d", &a);
getchar();                /* 吃掉上一次输入残留的换行符 */
scanf("%c", &b);

```

```
/* 方法 2: 在 %c 前加一个空格, 跳过前面所有空白字符 */
scanf("%d", &a);
scanf(" %c", &b);          /* 注意 %c 前面那个空格是关键 */
```

两种方法的差别在于：getchar() 只能吃掉**一个**字符，如果缓冲区里残留了多个空白（比如输入时多敲了几个空格），仍可能出错；而格式串中的空格表示“跳过任意多个空白字符”，更稳健。因此在混合读入数值与字符时，推荐统一写成 " %c"。

4.5 本章知识点小结

- 字符以 ASCII 码存储，与整数通用；大小写字母的码值相差 32；
- getchar()/putchar() 面向单个字符；getchar() 从输入缓冲区取字符，回车也进缓冲区；
- printf 由格式控制字符串与输出值参数表组成，格式符与参数类型、个数必须一一对应；域宽与精度写作 %m.nf；
- scanf 的参数是**地址**，必须有 &；数值型可用任意空白分隔，格式串中的普通字符必须原样输入；
- %c 不跳过空白，混合输入时用“空格 + %c”或 getchar() 清掉残留回车；
- double 的输入必须用 %lf，而输出用 %f 即可。

4.6 本章常见错误小结

表 4.3: 第 4 章常见错误与纠正

易错写法	错在哪里	纠正
scanf("%d", n);	漏写取地址符，数据写到非法地址	scanf("%d", &n);
scanf("%d\n", &n);	\n 表示跳过所有空白，程序会一直等待输入	去掉 \n
scanf("%f", &x) 而 x 是 double	类型不匹配，读入错误	用 %lf
scanf("%d,%d", &a,&b) 输入 12 34	格式串中的逗号必须原样输入	输入 12,34 或改格式串
连续 scanf("%d") 后跟 scanf("%c")	%c 读到残留的回车符	scanf(" %c", &ch)
printf("%d", 3.5)	格式符与参数类型不匹配	用 %f 输出实型
printf("%f", n) 而 n 是 int	同上，输出不可预期的值	换成 %d
putchar("a")	双引号是字符串，参数应为字符	putchar('a')

第 5 章

选择控制结构

5.1 生活中与计算机中的问题求解方法

人解决问题时，会根据情况随时调整做法；计算机不会，它只能执行事先写好的、每一步都明确的指令。因此在让计算机求解之前，必须先把“遇到什么情况就怎么做”想清楚并写成判定规则，这就是**选择结构**（Selection Structure）的由来。

结构化程序设计的观点是：任何复杂问题都可以只用**顺序、选择、循环**三种基本结构编程实现。本章讲第二种结构。

5.2 算法的概念及其描述方法

5.2.1 算法的概念

所谓**算法**（Algorithm），是解决问题的方法和步骤。一个算法必须具备以下特性：

表 5.1: 算法的五个特性

特性	含义
有穷性	一个算法必须能在执行有限个步骤之后终止，且每一步都能在有限时间内完成
确定性	每一步的含义必须确切无二义，相同的输入必得到相同的输出
可行性	每一步都能通过有限次基本运算实现
有零个或多个输入	算法可以没有输入（如直接打印固定图形），也可以有多个输入
有一个或多个输出	算法必须有结果，没有输出的算法没有意义

5.2.2 算法的描述方法

常用的描述方法有三种，各有适用场合：

实际做题时不必强求画图，但养成“先用自然语言写出判定步骤、再翻译成代码”的习惯，可以避免大量逻辑错误。N-S 图的价值在于：它用嵌套的框表示结构，画不出“随意跳转”，因此天然符合结构化程序设计的要求。

表 5.2: 算法描述方法对比

方法	优点	缺点
自然语言	易懂，适合口头交流	容易产生二义性、不够简洁
传统流程图	直观显示控制流向，判断用菱形、处理用矩形、输入输出用平行四边形	复杂算法时连线交叉多，不便于逐步求精
N-S 图（盒图）	用矩形框嵌套表示结构，取消流向线，强制结构化	修改与扩充不如流程图方便
伪代码	介于自然语言与程序之间，便于直接翻译成代码	无统一标准，不能直接运行

5.3 关系运算符与关系表达式

关系运算符用于比较两个操作数的大小关系，共 6 个：`<`、`<=`、`>`、`>=`、`==`、`!=`。由关系运算符及其操作数构成的表达式称为**关系表达式**。

（1）关系表达式的值只有两种：真为 1，假为 0。这是 C 语言与数学表达最不同的一点。例如 `5 > 3` 的值是 1，`5 < 3` 的值是 0，它们可以直接参与算术运算：`(5 > 3) + 1` 等于 2。

（2）关系运算符的优先级低于所有算术运算符。因此 `n % 2 != 0` 先算 `n % 2` 再与 0 比较；同理 `a + b > c` 先算加法。

（3）C 中“真”是“非 0”，“假”是“0”。由于这个约定，`n % 2 != 0` 与 `n % 2` 在逻辑上完全等价：`n` 为奇数时 `n % 2` 非 0 即为真。这种简化在判奇偶、判非空时很常用，但会降低可读性，需要权衡。

（4）判决等要用 ==，不是 =。`if (a = 5)` 是把 5 赋给 `a` 再判断 `a` 的值（永远为真），语法合法所以编译器不报错，是极难查的错误。良好的写法是把常量写在左边：`if (5 == a)`，一旦误写成 `if (5 = a)` 编译器立刻报错。

（5）浮点数不要直接用 == 比较。由于实数在内存中以浮点形式近似存储、精度有限，`0.1 + 0.2 == 0.3` 很可能为假。正确做法是判断二者之差是否小于一个足够小的数（如 10^{-6}）：

```
#define EPS 1e-6
if (fabs(a - b) <= EPS)       /* 认为 a 与 b 相等 */
```

> **核心考点** “实数与 0 比较”必须写成 `fabs(x) <= EPS`，不能写 `x == 0`。凡是看到程序里用 == 比较两个实型变量，几乎可以断定这里藏着 bug。

5.4 用于单分支控制的条件语句

单分支选择结构表示：条件成立时执行 A 操作，否则什么都不做。

```
if (表达式P)
    语句A;
```

执行逻辑：若表达式 P 的值为**真（非 0）**，则执行语句 A，然后继续执行 `if` 后面的语句；若为假，直接跳过语句 A。

使用要点：

- `if` 后面的表达式可以是任意类型的表达式，只要其值能判断为 0 或非 0；不必写成 `if (x == 1)`，写 `if (x)` 效果相同；
- `if` 与 `()` 之间不能加分号，否则 `if` 的“语句 A”变成了空语句，后面的语句永远执行；
- 分支里若有多条语句，必须用一对花括号（4.4 节讲到的复合语句）变成一条语句。

5.5 用于双分支控制的条件语句

双分支选择结构表示：条件成立执行语句 1，否则执行语句 2。

```
if (表达式P)
    语句1;
else
    语句2;
```

单分支与双分支的区别在语义上：单分支面临的选择是“执行一条语句，或者跳过它”；双分支面临的选择是“在两条语句中选一条执行”——两条路必走其一。因此双分支能保证**总有一个结果被处理**，这在“求两个数的最大值”之类的问题上省掉了预先赋值的麻烦。

```
if (a > b)
    max = a;
else
    max = b;       /* a <= b 的情况，无需再写判断 */
```

5.6 条件运算符和条件表达式

条件运算符 ？ ：是 C 语言中**唯一的三元运算符**，需要三个操作数。由它构成的表达式称为**条件表达式**：

```
表达式1 ? 表达式2 : 表达式3
```

含义是：若表达式 1 的值非 0，则该条件表达式的值为表达式 2 的值，否则为表达式 3 的值。上面求最大值的语句可以简化为一行：

```
max = a > b ? a : b;
```

三条注意：

1. 条件运算符的优先级低于关系运算符和算术运算符，高于赋值运算符，因此 `max = a > b ? a : b` 不需要额外括号；
2. 它是右结合的，`a ? b : c ? d : e` 等价于 `a ? b : (c ? d : e)`；
3. 用条件表达式改写时，两个分支必须是**表达式**（有值），不能是语句。若分支里要做多件事（如同时赋值和输出），只能老老实实用 `if-else`。

5.7 用于多分支控制的条件语句

多分支结构用 `else-if` 形式实现：

```
if (表达式1)        语句1
else if (表达式2)   语句2
else if (表达式m)   语句m
else                语句m+1
```

功能是：若表达式 1 为真，执行语句 1；否则若表达式 2 为真，执行语句 2；……若前面所有表达式都为假，执行最后的 `else` 分支。**各分支是互斥的**，一旦某条件成立，其余分支全部跳过——这一点决定了条件的书写顺序会直接影响结果。

复合语句与花括号。语法上每个分支只允许放**一条语句**，而实际操作往往需要多条，这时必须用一对花括号把这些语句括起来构成**复合语句**（Compound Statement）。复合语句在逻辑上是一个整体，可以被当作一条语句用在任何允许单条语句的位置。

```
if (a > b) {
    max = a;
    printf("max = %d\n", a);      /* 加花括号，两条语句都在分支内 */
} else {
    max = b;
    printf("max = %d\n", b);
}
```

忘记花括号的后果：编译器只把 `if` 后面的第 1 条语句当作分支体，第 2 条被放到分支之外，于是 `else` 找不到匹配的 `if`，编译时报告 `else without a previous 'if'`。所以推荐的做法是：**无论分支里有几条语句，都写成复合语句形式**。这样以后往里添加语句时不必再改结构。

悬空 else 的配对规则：`else` 总是与它**前面最近的、尚未配对的** `if` 配对，与缩进无关。

```c
if (a > 0)
    if (b > 0)  printf("both\n");
else                        /* 这个 else 配的是内层 if，不是外层！ */
    printf("a <= 0\n");
```

要让 else 配外层 if，必须给内层 if 加花括号。这类错误缩进看起来完全正确，但语义错了，是考试和调试中的高频陷阱。

5.8 用于多路选择的 switch 语句

当需要根据一个表达式的**多个确定取值**分别处理时，用 switch 比 else-if 链更清晰：

```c
switch (表达式) {
    case 常量表达式1:  语句序列1;  break;
    case 常量表达式2:  语句序列2;  break;
    ...
    default:           语句序列n;
}
```

规则与陷阱：

1. switch 后的表达式必须是**整型或字符型**（不能是实型、字符串）；
2. case 后面必须是**常量表达式**（如 1、'A'、2+3），不能是变量，且各 case 的值不能重复；
3. **break 不是语法必需，而是逻辑必需**。没有 break 时，程序会从匹配的 case 开始**一路往下执行**（俗称“穿透”，fall through），直到遇到 break 或 switch 结束。多个 case 共用同一段处理代码时，这种穿透是有意为之的写法：

```c
switch (ch) {
    case 'a': case 'e': case 'i': case 'o': case 'u':
        printf("vowel\n");  break;      /* 有意穿透，共用一段代码 */
    default:
        printf("consonant\n");
}
```

4. default 可以放在任意位置，通常写在最后；它的作用是“以上 case 都不匹配时执行”；
5. switch 的每个分支体内可以定义变量，但若在 case 后直接定义，作用域会跨越后续 case，容易出错，建议加花括号限制作用域。

> **核心考点** 判断该用 switch 还是 else-if：如果条件是“某个表达式的值等于若干离散常量”之一，用 switch；如果条件是**区间或复合条件**（如 score >= 90、x > 0 && y > 0），switch 表达不了，必须用 else-if。

5.9 逻辑运算符和逻辑表达式

逻辑运算符有 3 个：!（逻辑非）、&&（逻辑与）、||（逻辑或）。由它们构成的表达式称为逻辑表达式，其值**只有 0 和 1 两种**。

表 5.3: 逻辑运算真值表

a	b	!a	a && b	a || b	说明
0	0	1	0	0	都假才假（与）、有真即真（或）
0	非 0	1	0	1	
非 0	0	0	0	1	
非 0	非 0	0	1	1	

（1）优先级由高到低为：

! > 算术运算符 > 关系运算符 > && > || > 赋值运算符

因此 `a > b && c > d` 不需要加括号；而 `!a == b` 是 `(!a) == b`，若想表达“a 不等于 b”必须写 `a != b` 或 `!(a == b)`。

（2）短路求值：`a && b` 中若 a 为假，b 不再求值；`a || b` 中若 a 为真，b 不再求值。这既是效率优化，也常被用来写“安全的条件”：

```
if (n != 0 && 10 / n > 2)      /* n 为 0 时不会执行除法，避免除零 */
```

反之要警惕副作用：`i > 0 && i++` 中 `i++` 可能根本不执行。

（3）数学上的连写不能照搬。`0 < x < 10` 在 C 中先算 `0 < x` 得 0 或 1，再用这个结果与 10 比较，结果恒为真，逻辑完全错了。正确写法是 `x > 0 && x < 10`。

（4）区分逻辑运算符与位运算符：`&&` 是逻辑与（结果为 0 或 1，有短路），`&` 是按位与（逐位运算，无短路）。`if (a & b)` 与 `if (a && b)` 在 a、b 都非 0 时结果可能一致，但语义完全不同，混用是隐蔽的 bug 源。

5.10 本章扩充内容

5.10.1 程序测试

程序写完必须测试，目的是“证明它有错”而不是“证明它没错”。常用两类方法：

- **白盒法**（结构测试）：依据程序的内部逻辑设计测试用例，追求语句覆盖、分支覆盖——每条语句至少执行一次，每个判定的真、假两个分支都至少走一次；
- **黑盒法**（功能测试）：不看内部结构，只依据功能说明设计输入，重点考察**边界值**（最大值、最小值、0、空输入）和异常输入。

对选择结构而言，最容易漏测的是“所有条件都不成立”这条路径，以及边界上的等号（`>=` 写成 `>` 时恰好取等值才能发现）。因此每个 `if` 都应准备至少一组“使条件为真”和一组“使条件为假”的数据。

5.10.2 对输入非法字符的检查与处理

用户输入不受程序控制，防御性程序设计要求对输入做检查。最基本的两条：

```
int n;
if (scanf("%d", &n) != 1) {       /* 返回值是成功读入的数据个数 */
    printf("input error\n");
    return 1;                      /* 或 exit(1) 直接终止 */
}
if (n < 0 || n > 100) {            /* 检查取值范围 */
    printf("out of range\n");
    return 1;
}
```

若输入流中残留了非法字符，可先用 `getchar()` 逐个读掉，或清空缓冲区后再重新读入。要点是：**任何来自键盘的数据都必须当作“可能不合法”处理**，而不是假定用户会按提示输入。

5.10.3 位运算符

位运算符直接对整数的二进制位操作，共 6 个：

表 5.4: 位运算符及其典型用途

运算符	名称	典型用途
`&`	按位与	取某几位（配合掩码）、清零特定位
`|`	按位或	将某几位置 1
`^`	按位异或	翻转特定位；不借助临时变量交换两数
`~`	按位取反	单目运算符，逐位取反
`<<`	左移	左移 n 位相当于乘 2^n（可能溢出）
`>>`	右移	右移 n 位相当于除以 2^n（对无符号数）

```
unsigned char x = 0x5A;          /* 0101 1010 */
x & 0x0F                         /* 0x0A: 取出低 4 位 */
x | 0x01                         /* 0x5B: 把最低位置 1 */
x ^ 0xFF                         /* 0xA5: 全部翻转 */
a ^= b; b ^= a; a ^= b;          /* 交换两个整数，不需要中间变量 */
```

位运算的前提是**先想清楚每个数的二进制形式**。它与逻辑运算的区别必须分清：`!`、`&&`、`||` 把操作数当作“真/假”，结果为 0 或 1；`~`、`&`、`|`、`^` 把操作数当作“位串”，逐位运算。

5.11 本章知识点小结

- 算法是解决问题的方法和步骤，必须满足有穷性、确定性、可行性、有输入输出；
- 关系表达式的值是 1 或 0；关系运算符优先级低于算术运算符；实型比较要用 `fabs(x-y) <= EPS`；
- 三种条件语句：单分支 `if`、双分支 `if-else`、多分支 `else-if`；
- 条件运算符 `？ ：`是唯一的三元运算符，右结合，只能用于表达式；
- 多分支里只要分支含多条语句就必须加花括号；`else` 与最近的未配对 `if` 配对；
- `switch` 的表达式须为整型或字符型，`case` 后须为常量且互不重复，通常每个 `case` 后要 `break`；
- 逻辑运算结果为 0/1，有短路特性；`0 < x < 10` 这类数学连写在 C 中是错的。

5.12 本章常见错误小结

表 5.5: 第 5 章常见错误与纠正

易错写法	错在哪里	纠正
`if (a = 5)`	赋值而非比较，条件恒真	`if (a == 5)` 或 `if (5 == a)`
`if (x == 0.0)`	浮点数是近似值，相等判断不可靠	`if (fabs(x) <= EPS)`
`if (a > b);`	多写分号，分支变成空语句	去掉分号
`if (0 < x < 10)`	先得 0/1 再与 10 比较，恒真	`x > 0 && x < 10`
`else` 缺花括号	编译器只认第 1 条语句，导致 `else without previous if`	分支统一加花括号
`switch` 中漏 `break`	穿透执行后续 `case`	每个分支末尾加 `break`
`case x:`（x 为变量）	`case` 后必须是常量表达式	改用 `else-if`
`a & b` 代替 `a && b`	按位与不是逻辑与，无短路	用 `&&`

第6章

循环控制结构

6.1 循环控制结构与循环语句

实际应用中的许多问题都涉及**重复执行**一些操作，如级数求和、穷举或迭代求解。若重复处理的次数是已知的，称为**计数控制的循环**（Counter Controlled Loop）；若次数未知、由给定条件控制，称为**条件控制的循环**（Condition Controlled Loop）。两者都需要用循环结构实现。

循环结构有两种类型：

- **当型循环**：先判断条件 P，成立（真）时反复执行 A 操作，直到条件不成立（假）时结束——**先测后做**；
- **直到型循环**：先执行 A 操作，再判断条件 P，成立则继续反复执行 A，直到条件不成立——**先做后测**。

这个差别直接决定了循环体**最少执行几次**：当型可能一次都不执行，直到型至少执行一次。

C 语言提供 `while`、`do-while`、`for` 三种循环语句来实现循环结构。被重复执行的语句序列称为**循环体**（Body of Loop）。

6.1.1 while 语句

`while` 属于**当型**循环：

```
while (循环控制表达式)
{
    语句序列;       /* 循环体 */
}
```

执行过程：

1. 计算循环控制表达式的值；
2. 若为真，执行循环体语句，然后返回步骤 1；
3. 若为假，退出循环，执行循环体后面的语句。

循环控制表达式在**执行循环体之前**测试，因此若第一次判断就为假，循环体一次也不执行。

6.1.2 do-while 语句

do-while 属于**直到型**循环，注意结尾的分号不能少：

```
do
{
    语句序列;       /* 循环体 */
} while (循环控制表达式);
```

与 while 不同的是，do-while 的循环控制表达式在**执行循环体之后**测试，因此**循环体内的语句至少被执行一次**。这正是“先输入再校验”这类场景需要的语义：至少让用户输一次，才能判断输入是否合法。

6.1.3 for 语句

for 也属于当型循环，是 C 程序中使用频率最高的循环语句：

```
for (初始化表达式; 循环控制表达式; 增值表达式)
{
    语句序列;       /* 循环体 */
}
```

三个表达式的分工：

- **初始化表达式**：为循环控制变量赋初值，决定循环的起始条件；
- **循环控制表达式**：控制循环继续执行的条件，值为真（非 0）时继续，否则结束循环；
- **增值表达式**：每执行完一次循环体后对循环控制变量做一次变化，决定循环何时结束。

每轮（包括第一轮）循环体执行之前都要测试一次循环控制条件，每轮循环体执行完后都要执行一次增值表达式。for 语句与下面形式的 while 语句等价：

```
初始化表达式;
while (循环控制表达式)
{
    语句序列;
    增值表达式;
}
```

三条书写规则：

1. 三个表达式之间的分隔符是**分号**，有且仅有两个分号，不能多也不能少；
2. 循环控制表达式一般不要省略，若省略则表示循环条件恒真（死循环）。若已在 for 前面赋过初值，初始化表达式可省略；若已在循环体中改变了循环控制变量，增值表达式可省略。因此 for (; cond;)、for (i=0; cond;) 都是合法的；

3. 即使循环体只有一条语句，也建议用花括号括起来。否则日后往循环体里加语句时，只有第一条在循环内，会造成逻辑错误。

```
/* 三种语句实现同一件事：累加 1 到 n */
sum = 0;
for (i = 1; i <= n; i++)  sum += i;

sum = 0;  i = 1;
while (i <= n) { sum += i; i++; }

sum = 0;  i = 1;
do { sum += i; i++; } while (i <= n);
```

6.2 计数控制的循环

计数控制循环的典型应用是**累加求和**与**累乘求积**。以计算 $1+2+\cdots+n$ 为例，如果为每个加数定义一个变量，n 未知时就无法确定变量个数，n 很大时又浪费空间。改用循环后，只需三个变量：循环控制变量 `i`、累加和 `sum`、循环次数 `n`。

（1）累加变量必须初始化为 0，累乘变量必须初始化为 1。这是最容易被忽略、后果最严重的一步。删掉 `sum = 0` 后，`sum` 里是残留的垃圾值，累加结果会变成 `4199309` 之类的乱数——因为“在垃圾值的基础上加”。

（2）`sum = sum + i` 为什么能累加？因为赋值号右侧的 `sum` 执行的是**读**操作，左侧的 `sum` 执行的是**写**操作：先把 `sum` 的旧值读出来，与 `i` 相加，再写回 `sum`。所以每轮循环都“在上一次的基础上继续累积”。

（3）循环变量的出口值要注意。对 `for (i = 1; i <= n; i++)`，循环结束时 `i` 的值是 $n+1$ 而不是 n，因为最后一轮循环体执行完后还要执行一次增值表达式才做判断。

（4）找通项规律是计数循环的核心。当被累加的各项不是简单的 i，而是 i^2、$\frac{1}{i}$、$(-1)^i\frac{1}{2i-1}$ 等形式时，关键是写出**第 i 项**的表达式，并尽量用**递推**的办法由上一项算出下一项，而不是每轮都从头计算：

```
/* 累乘求阶乘：term 表示当前项，每轮由上一项递推 */
double term = 1.0, result = 0.0;
for (i = 1; i <= n; i++) {
    term = term * i;        /* 第 i 项 = 第 i-1 项 * i */
    result = result + term;
}
```

（5）循环次数的确定。判断循环执行多少次，看的是“初值、终值、步长”三件事：`for (i = 1; i <= n; i++)` 执行 n 次；`for (i = 0; i < n; i++)` 也执行 n 次；而 `for (i = 1; i < n; i++)` 只执行 $n-1$ 次。边界上的等号是“差一错误”（off-by-one）的根源。

6.3 嵌套循环

一个循环体内又包含另一个完整的循环结构，称为**嵌套循环**。外层循环每执行一次，内层循环就要完整地执行一遍，因此总执行次数是**各层次数之积**。以打印图形为例：

```
for (i = 1; i <= rows; i++) {          /* 外层：控制行数 */
    for (j = 1; j <= i; j++)           /* 内层：控制本行字符数 */
        putchar('*');
    putchar('\n');                      /* 每行结束换行 */
}
```

嵌套循环要注意三点：

1. **内外层循环变量不能同名**。同名会互相覆盖，导致内层破坏外层的计数，常常直接造成死循环或次数错乱；
2. **内层循环的初值必须每轮重新设置**。若把 `j` 的初始化写到外层循环之外，第二轮起 `j` 已经是旧值，内层可能一次都不执行；
3. **花括号要成对**，尤其在内层循环体只有一条语句时，漏写花括号会让“每行换行”的语句跑进内层，输出变成一列。

6.4 条件控制的循环

当循环次数事先未知、由某个条件决定何时结束时，用条件控制的循环。典型场景是“反复读入数据，直到满足某条件为止”：

```
/* 猜数游戏：反复猜，直到猜中 */
do {
    printf("Please guess: ");
    scanf("%d", &guess);
    count++;
    if (guess > target)      printf("too big\n");
    else if (guess < target) printf("too small\n");
} while (guess != target);   /* 猜中即结束 */
printf("You got it in %d times\n", count);
```

这类循环有两条设计要点：

- **循环控制条件必须在循环体内被改变**，否则一旦进入就是死循环。上面的 `guess` 每轮由 `scanf` 更新，`count` 每轮递增，条件终会成立；
- 若“至少要做一次”才谈得上判断（如先读入才能判断是否结束），用 `do-while`；若“可能一次都不做”，用 `while`。

6.5 流程的转移控制

goto、break、continue 和 return 是 C 语言中用于控制流程转移的跳转语句，其中 return 用于从函数返回值，在第 7 章介绍。

6.5.1 goto 语句

goto 是**无条件转向**语句，既可向下跳转，也可往回跳转：

```
goto 语句标号;
...
语句标号: 语句;
```

语句标号（Label）代表跳转的目标位置，必须用合法标识符表示，命名规则与变量名相同。goto 通常与 if 联合使用，构成“满足条件才跳”的形式。

良好的编程风格要求**少用、慎用 goto**，尤其不要用往回跳转的 goto，也不要用它制造永远不会被执行的死代码。结构化程序设计规定：程序中**尽量不使用多于一个 goto 语句标号**，且只允许在“单入口单出口”的模块内**向前**跳转，不允许回跳。

goto 真正合适的情形只有两种：一是从多层嵌套循环中一次性跳出，二是跳向共同的出口位置做统一的错误处理。

6.5.2 break 语句

break 除用于退出 switch 结构外，还可用于 while、do-while、for 的循环体中。执行到 break 时，**循环立即终止**，从循环语句后的第一条语句继续执行。

它是“有条件跳转”，且跳转位置被唯一限定为**紧接循环语句后的第一条语句**。正因为位置固定，它不需要标号，也就避免了因标号过多导致的流程混乱。

关键限制：在嵌套循环中，break **只对包含它的最内层循环起作用**，不能一次跳出多重循环。要跳出 k 层，只能逐层 break，或者用 goto。

6.5.3 continue 语句

continue 与 break 都用于循环内部控制，但效果不同：遇到 continue 时，程序**跳过本次循环中后面尚未执行的语句，开始下一次循环**——只结束本次循环，并不终止整个循环。

```
for (i = 1; i <= 5; i++) {
    scanf("%d", &n);
    if (n < 0) {
        printf("negative, skipped\n");
        continue;            /* 不显示负数，但继续读下一个 */
    }
    printf("n = %d\n", n);
```

```
}
```

一个必须记住的差异：`for` 循环里的 `continue` 会**执行增值表达式**再判断条件；而 `while` 循环里的 `continue` 会**直接跳去判断条件**，若增值语句写在 `continue` 之后就被跳过，从而造成死循环。所以“把 `for` 改写成 `while`”在有 `continue` 时二者**并不等价**——这是教科书反复提醒的一点。

6.5.4 穷举法编程实例

穷举（Exhaustion）就是通过尝试问题的所有可能来得到最终答案。以“韩信点兵”为例：按 1 至 5 报数末位为 1，按 1 至 6 报数末位为 5，按 1 至 7 报数末位为 4，按 1 至 11 报数末位为 10，求至少有多少兵。设兵数为 x，则需

$$x\%5 = 1 \;\&\&\; x\%6 = 5 \;\&\&\; x\%7 = 4 \;\&\&\; x\%11 = 10$$

从 1 开始逐个试验，第一个使条件成立的 x 即为所求：

```
int x = 1;
while (!(x % 5 == 1 && x % 6 == 5 && x % 7 == 4 && x % 11 == 10))
    x++;
printf("x = %d\n", x);
```

穷举法有三个必须交代清楚的问题：

1. **枚举范围要覆盖解**。若把上界写成 `x < 3000`，而真正的解在范围之外，程序将“一无所获”。偶然得到正确答案不算对，那是瞎猫碰到死耗子；
2. **要有终止机制**。若去掉上界限制，虽然一定能找到解，但会得到不止一个解，而且找完第一个解后循环还会继续，最终陷入死循环。要么用 `break` 在找到后立即退出，要么在条件里体现“至少”的含义；
3. **枚举量要尽量小**。可以借助数学性质剪枝（如把步长从 1 改为 11 的倍数），在解空间很大时这一点决定程序能否跑完。

6.6 本章扩充内容

6.6.1 结构化程序设计的核心

结构化程序设计的核心是：**任何复杂问题都只用顺序、选择、循环三种基本结构来编程实现**，并且每个模块保持“单入口、单出口”。由此派生出几条具体规定：

- 限制和避免使用 `goto`，最多一个标号，只允许向前跳；
- 采用自顶向下、逐步求精的模块化方法分解问题（第 7 章）；
- 程序结构的好坏不由“有没有 `goto`”决定，避免 `goto` 只是手段而非目的。

6.6.2 常用的程序调试与排错方法

调试的目标不是“用工具乱翻”，而是**先想清楚，再验证**。几条实用做法：

- **插入打印语句**往往比单步跟踪更高效：在关键位置打印中间变量，观察值是否符合预期。排错语句留在程序中，可反复观察；单步执行则转瞬即逝；
- **增量测试**：扩充程序时不断测试新加入的代码及它与原有可运作单元的联系，把待调试的代码限制在很小的范围内，从而缩小查错区域；
- **改错三步**：先想清楚修改这一行会不会引发其他缺陷；乘胜追击，改正所有同类 bug；改完立即做**回归测试**（对以前修过的 bug 重新测试），确认没有引入新问题；
- **重视错误现象**。错误结果常常提供线索：问题为什么发生？以前是否遇到类似情况？哪几条语句刚被修改过？引起错误的输入有什么特殊之处？

6.6.3 类型溢出

C 语言提供的每种类型都有取值范围，向其中赋超过范围的值就会产生**类型溢出**（Type Overflow），得到错误结果。典型例子是“计算自然数的立方和，直到立方和大于等于 1000000”：

```
short i, sum = 0;               /* 错误：short 只有 2 字节 */
for (i = 1; ; i++) {
    sum = sum + i * i * i;
    if (sum >= 1000000) { printf("count = %d\n", i); break; }
}
```

`short` 只占 2 字节，取值范围是 $-32768 \sim 32767$，而 1000000 早已超出，`sum` 会溢出成负数，条件永远不成立，于是**死循环**。正确结果应为 45 项，把变量改为 `int` 或 `long` 即可。

溢出还有两种更隐蔽的形式：

- **中间结果溢出**：即便最终结果在范围内，中间乘积也可能越界。例如 `n * n` 中 `n` 为 `int` 时，n 稍大就会溢出，此时应先强转为 `double` 再乘；
- **整数除法丢精度**：计算 $\frac{n^2}{(n-1)(n+1)}$ 时，若 `n` 为 `int`，`n*n / ((n-1)*(n+1))` 是整数除法，小数部分被舍去，必须写成 `(double)(n*n) / ((double)(n-1)*(n+1))`。

教训是：**预先估算运算结果的可能取值范围，选用取值范围足够大的类型**；把高位的平台移植到低位平台时尤其要重新检查。

6.7 本章知识点小结

- 循环分当型（`while`、`for`，先测后做）与直到型（`do-while`，先做后测，至少执行一次）；
- `for` 的三个表达式由两个分号分隔；初始化、增值可省略，条件省略即死循环；

- 累加变量初值为 0，累乘变量初值为 1；`sum = sum + i` 的读写次序是理解累加的关键；
- 嵌套循环总次数为各层次数之积；内外层变量不能同名，内层初值每轮必须重置；
- `goto` 可任意跳转但破坏结构；`break` 终止整个循环且只跳出最内层；`continue` 只结束本次循环；
- `for` 中的 `continue` 会执行增值表达式，`while` 中不会，二者不一定等价；
- 穷举法要保证枚举范围覆盖解、有终止机制，并尽量缩小枚举空间；注意类型溢出与整数除法丢精度。

6.8 本章常见错误小结

表 6.1: 第 6 章常见错误与纠正

易错写法	错在哪里	纠正
累加前未 `sum = 0`	在垃圾值上累加，输出乱数	循环前初始化
`for (i = 1; i <= n; i++);`	圆括号后多写分号，循环体变成空语句	去掉分号
`while (cond);`	同上，且容易变成死循环	去掉分号
循环体中未改变控制变量	条件永远成立，死循环	循环体内更新变量
`for (i=1, j=1; i<n; i++, j++)`	三个表达式之间用逗号而非分号	用两个分号分隔三部分
`do {...} while (cond)`	漏写结尾分号，编译错误	`while (cond);`
`break` 想跳出两层循环	`break` 只作用于最内层	逐层 `break` 或用 `goto`/`return`
`short sum` 累加大数	类型溢出，结果为负，循环不终止	用 `int`/`long`

第 7 章

函数与模块化程序设计

7.1 分而治之与信息隐藏

当程序规模变大时，把所有代码都堆在 `main` 里会带来两个致命问题：一是人脑一次只能处理有限的信息量，逻辑一复杂就看不懂；二是同一段代码在多处重复，改一处要改很多处。

解决思路有两条，它们共同构成函数式设计的出发点：

- **分而治之**（Divide and Conquer）：把一个大问题分解成若干个相对独立、规模更小的子问题，每个子问题用一个**函数**实现；只要子问题都解决了，大问题就解决了。
- **信息隐藏**（Information Hiding）：函数的调用者只需要知道它“做什么”（接口），不必知道它“怎么做”（实现）。函数内部用什么变量、走什么流程，对外部不可见，也不会被外部干扰。

信息隐藏带来的直接好处是：函数内部改了实现，只要接口不变，调用它的代码一行都不用改。这正是“可维护性”的含义。

7.2 函数的定义

7.2.1 函数的分类

表 7.1: 函数的分类

分类依据	类型	说明与实例
来源	库函数	由 C 标准库提供，已编译好，包含头文件即可用，如 `printf`、`sqrt`
	用户自定义函数	为解决具体问题自己编写
有无返回值	有返回值	返回类型为具体类型，如 `int`、`double`，用 `return` 返回结果
	无返回值	返回类型写 `void`，只完成某种操作（如输出、修改数组）
有无参数	有参函数	通过形参接收外部数据
	无参函数	形参表写 `void` 或留空，如 `int main(void)`

7.2.2 函数的定义

函数定义的一般形式为

```
返回类型  函数名(形式参数表)
{
    函数体
}
```

各部分的规定：

1. **返回类型**指明函数返回值的类型，可以是 int、double、char、指针等；不返回值的写 void；
2. **函数名**是合法标识符，应能表达函数的功能（如 Max、IsPrime）；
3. **形式参数表**中每个形参必须**逐个指明类型**，写成 (int a, int b) 或 (int a, b) 都可，但 (a, b) 不行。无形参时写 (void) 或 ()；
4. **函数体**包含声明部分与语句部分，声明必须写在可执行语句之前。

```
int Max(int a, int b)          /* 返回值类型 + 函数名 + 形参表 */
{
    int m;                     /* 声明部分 */
    m = a > b ? a : b;         /* 语句部分 */
    return m;                  /* 返回结果 */
}
```

一条常被忽略的限制：**C 语言不允许在函数内部再定义函数**（不能嵌套定义），所有函数都是平行的、独立的。因此“把函数写在 main 里面”是非法的。

7.3 向函数传递值和从函数返回值

7.3.1 函数调用

调用形式为

函数名 (实际参数表);

实参可以是常量、变量或表达式，其个数与类型应与形参一一对应。

(1) 参数传递是“值传递”。调用时，实参的值**单向**复制给形参，形参在函数内部得到了一个**副本**。因此在函数里修改形参，不会影响调用者的实参：

```
void TryChange(int x) { x = 100; }      /* 改的只是副本 */

int main(void)
{
```

```
    int a = 5;
    TryChange(a);
    printf("%d\n", a);                    /* 仍然是 5 */
    return 0;
}
```

这是“按值调用”（Call by Value）的语义。第 9 章会讲用指针作参数实现“模拟按引用调用”，第 8 章会讲用数组作参数（数组名退化为地址，因此函数内修改数组元素会影响调用者）——这两者是值传递规则下的特例，不是例外。

（2）调用过程。执行到函数调用时，系统会：为被调函数分配存储空间（栈帧），把实参值复制给形参，把控制权交给被调函数；函数执行到 `return` 或函数体结束时，把返回值带回调用处，释放栈帧，控制权交回。理解这个顺序，才能解释 7.5 节的生存期问题。

（3）返回值。`return` 语句有两种用法：`return 表达式;` 返回一个值；`return;` 用于 `void` 函数，表示提前结束。函数遇到 `return` 立即返回，其后的语句不再执行。返回值的类型若与函数声明的返回类型不一致，会按赋值规则自动转换。

7.3.2 函数原型

函数原型（Function Prototype）即函数声明，只给出函数的“外貌”，不含函数体：

```
int Max(int a, int b);          /* 原型：有参数名 */
double Sqrt(double);            /* 原型：可只写类型，参数名可省 */
void PrintMenu(void);
```

为什么需要原型？因为 C 编译器**自上而下**编译源程序，遇到函数调用时：

- 若已经见过该函数的定义或原型，就能检查实参的个数、类型是否匹配，并按正确的方式传递参数、处理返回值；
- 若没见过，编译器只能**猜测**（旧标准按默认规则处理），可能导致参数传递方式错误、返回值解释错误，产生难以定位的运行时错误。

因此**先声明、后使用**是基本要求。一般把所有函数原型集中写在源文件开头，或写进自己的头文件，供多个源文件共享。

7.3.3 函数封装与防御性程序设计

封装是指把实现细节藏在函数内部，只通过参数和返回值与外界交互。**防御性程序设计**是指在函数入口对参数做合法性检查，不假定调用者一定传对：

```
double SafeDivide(double a, double b, int *ok)
{
    if (fabs(b) < 1e-12) {          /* 检查参数：除数为 0 */
        *ok = 0;
```

```
        return 0.0;                 /* 用返回码告知调用者失败 */
    }
    *ok = 1;
    return a / b;
}
```

常见的防御手段有三种：

1. **参数检查**：检查范围（数组下标、分母是否为 0、指针是否为空）、检查类型转换是否会溢出；
2. **用返回值表示错误**：返回 0 / −1 / NULL 等特殊值，或额外用一个参数带回状态；
3. **及早返回**：发现非法输入立即 return，避免错误值继续向下传播——错误越晚暴露，越难定位。

7.3.4 函数设计的基本原则

表 7.2: 函数设计的基本原则

原则	具体要求
单一职责	一个函数只做一件事。若函数名里出现“和”“并且”，通常该拆开
接口清晰	参数不要过多（一般不超过 4–5 个）；名字与参数含义明确，不靠注释解释
高内聚低耦合	函数内部各部分紧密相关；函数之间尽量少互相依赖
少用全局变量	通过参数传入、通过返回值传出，避免隐式的数据通路
副作用明确	函数要么返回结果，要么完成明确的操作，不要在“求值”的函数里偷偷输出
长度适中	一个函数以能在一屏内读完为宜，过长则拆分

7.4 函数的递归调用和递归函数

函数直接或间接地调用自身，称为**递归调用**；含递归调用的函数称为**递归函数**。

写递归函数必须同时具备两个要素，缺一不可：

1. **递归关系**：把原问题化为规模更小的同类问题，即“n 的解用 $n-1$ 的解表示”；
2. **终止条件**（base case）：规模小到可以直接求解时的出口，否则递归永不停止。

```
long Fact(int n)                    /* 求 n! */
{
    if (n == 0 || n == 1)           /* 终止条件 */
        return 1;
    return n * Fact(n - 1);         /* 递归关系 */
}
```

执行过程可以这样看：`Fact(4)` 展开成 $4 \times$ `Fact(3)`，再展开成 $4 \times 3 \times$ `Fact(2)`……直到 `Fact(1)` 返回 1，然后逐层“回归”把结果乘回去。每一层的形参 `n` 和返回地址都保存在**栈**上，因此递归深度越大，占用的栈空间越多；深度过大时会栈溢出。

递归与迭代（循环）的关系是：

- 理论上任何递归都可用循环加栈改写，任何循环也可用递归实现；
- 递归的**优点是表达简洁**，代码与数学定义几乎一一对应（如阶乘、斐波那契数列、汉诺塔）；
- 递归的**缺点是有函数调用开销、占栈空间**，且朴素递归可能重复计算（如 `Fib(n)` 直接递归的时间复杂度是指数级）。

因此：能用循环简单实现的（如累加、累乘）就用循环；问题本身具有递归结构、用循环写起来很别扭的，才用递归。

> **核心考点** 写递归先写终止条件，再写递归式，最后才考虑效率。忘记终止条件（或终止条件写错）会导致无限递归，典型症状是程序崩溃并提示栈溢出（stack overflow）。

7.5 变量的作用域和生存期

作用域回答“这个变量在程序**哪些地方**可以被使用”，生存期回答“这个变量的存储空间**什么时候**存在”。两者是不同的概念。

7.5.1 变量的作用域

表 7.3: 按作用域划分的变量

种类	定义位置	可见范围
局部变量	函数内部（或复合语句内部）	从定义处到所在函数（或块）结束；块内定义的只在块内可见
全局变量	所有函数之外	从定义处到本源文件结束；其他文件需用 `extern` 声明
函数原型中的参数名	函数声明中	仅在该声明中有效，可与定义中的名字不同

两条规则要记牢：

- **就近遮蔽**：若局部变量与全局变量同名，在局部变量的作用域内，全局变量被“遮蔽”，访问到的是局部变量。这极易造成“明明改了全局变量却没生效”的困惑，应通过命名区分（如全局加前缀 `g_`）来避免；
- 形参也是局部变量，其作用域就是整个函数体。

7.5.2 变量的生存期

按存储类别，变量的生存期分为四种：

表 7.4: 变量的存储类别与生存期

存储类别	关键字	生存期与特点
自动变量	`auto`（默认，可省略）	进入作用域时分配（栈上），离开时释放；未初始化则为垃圾值
静态变量	`static`	程序整个运行期间都存在；**只初始化一次**，函数返回后值保留
寄存器变量	`register`	建议编译器放入寄存器以提高访问速度（现代编译器一般自动优化）
外部变量	`extern`	声明“该变量在别处定义”，用于跨文件共享全局变量

静态局部变量是最容易考也最容易用错的一种，它兼具两种性质：

```
void Counter(void)
{
    static int count = 0;       /* 只初始化一次 */
    count++;
    printf("%d\n", count);
}
/* 连续调用三次，输出 1、2、3 —— 而不是 1、1、1 */
```

它与全局变量的区别只在**作用域**：静态局部变量仍只在函数内可见（外部无法访问），但生存期与全局变量相同。它适合保存“跨调用需要记住”的状态，但会让函数的行为依赖于调用历史，破坏可测试性，应谨慎使用。

7.6 模块化程序设计

7.6.1 模块分解的基本原则

模块是功能相对独立、接口清晰的一段程序（通常是一个或一组函数）。分解时遵循：

- **高内聚**：一个模块内部各部分的联系要紧密，共同完成一件明确的事；
- **低耦合**：模块之间尽量少依赖，接口参数尽量少，避免共享全局变量；
- **信息隐藏**：模块内部的实现细节不对外暴露，只暴露必要的接口；
- **规模适中**：模块过大难以理解，过小则接口开销与调用层次过多。

7.6.2 自顶向下、逐步求精

自顶向下（Top-down）是指从问题的总体目标出发，先写出主程序的框架（每一步对应一个函数调用），再逐层细化每个函数；**逐步求精**（Stepwise Refinement）是指每一层都只处理当前层次的细节，把更细的工作留给下一层。

```c
int main(void)
{
    ReadData();       /* 第 1 层：只写“做什么”，不写“怎么做” */
    Sort();
    Output();
    return 0;
}
```

这样写出来的 `main` 本身就是一份“程序说明书”，读者不必钻进细节就能看懂整体流程。

7.6.3 模块化程序设计与多文件编程

当程序由多个源文件组成时，约定的分工是：

表 7.5: 多文件编程的文件分工

文件	放什么	说明
头文件 `.h`	函数原型、宏定义、类型定义、`extern` 声明	供其他文件 `#include`，只放“声明”不放“定义”
源文件 `.c`	函数定义、内部使用的静态变量	编译成目标文件后由链接器连接
主文件	`main` 函数	调用各模块

```c
/* mymath.h —— 只放声明 */
#ifndef MYMATH_H
#define MYMATH_H
int    Max(int a, int b);
double CircleArea(double r);
#endif

/* mymath.c —— 放定义 */
#include "mymath.h"
int Max(int a, int b) { return a > b ? a : b; }
```

几个关键点：

- `#include <stdio.h>` 用尖括号，编译器到系统头文件目录查找；`#include "mymath.h"` 用双引号，先在当前目录查找——自定义头文件必须用双引号；

- 头文件只放**声明**。在头文件里定义变量或函数体，被多个源文件包含时会产生“重复定义”的链接错误；
- 若某函数或全局变量只在本文件内使用，加 `static` 修饰，可限制其可见性，避免命名冲突；
- 头文件保护（`#ifndef/#define/#endif`）防止同一头文件被重复包含。

7.6.4 条件编译

条件编译让编译器**按条件选择**参与编译的代码段，常用形式有三种：

```
/* 形式 1：常量表达式为真才编译 */
#if 表达式
    程序段 1
#elif 表达式
    程序段 2
#else
    程序段 3
#endif

/* 形式 2：是否定义过某宏 */
#ifdef DEBUG
    printf("debug: x = %d\n", x);
#endif

/* 形式 3：头文件保护（最常用） */
#ifndef MYMATH_H
#define MYMATH_H
/* 声明 */
#endif
```

典型用途：调试信息的开关（定义 `DEBUG` 才编译打印语句，发布时去掉即可）、不同平台的适配、头文件防重复包含。与 `if` 语句的本质区别是：条件编译在**预处理阶段**决定代码是否进入编译，不满足条件的代码根本不会生成目标代码；而 `if` 语句在运行时判断。

7.7 本章扩充内容——代码风格

代码风格不影响程序能否运行，却直接决定别人（和三个月后的自己）能否看懂。基本要求：

1. **缩进一致**：每层缩进统一用 4 个空格或一个制表符，不要混用；同一层的语句左边界对齐；
2. **一行一句**：不要在一行里塞多条语句，也不要把一条语句拆得七零八落；
3. **命名有意义**：`sum`、`count`、`maxValue` 比 `a`、`b`、`x1` 好得多；常量全大写，全局变量加前缀；
4. **注释解释“为什么”而不是“是什么”**：`i++; /* i 加 1 */` 是废话；`/*` 跳过行尾的换行符 `*/` 才有价值；

5. **避免深层嵌套**：嵌套超过三层就该考虑拆函数或提前返回；
6. **花括号成对且不省略**：即使分支或循环体只有一条语句，也写上花括号。

7.8 本章知识点小结

- 函数是分而治之与信息隐藏的载体：调用者只需知道接口，不必知道实现；
- 函数定义由返回类型、函数名、形参表和函数体组成；C 不允许函数嵌套定义；
- 参数传递是**值传递**，形参是实参的副本，函数内改形参不影响实参；
- 函数原型让编译器在调用处就能检查参数类型与个数，应先声明后使用；
- 递归函数必须同时有递归关系与终止条件；递归简洁但占栈、可能有重复计算；
- 作用域决定“哪里能用”，生存期决定“何时存在”；`static` 局部变量只初始化一次且值保留；
- 多文件编程：`.h` 放声明、`.c` 放定义；头文件要加保护；文件内私有函数用 `static`；
- 条件编译在预处理阶段选择代码，与运行时的 `if` 语句本质不同。

7.9 本章常见错误小结

表 7.6: 第 7 章常见错误与纠正

易错写法	错在哪里	纠正
在 `main` 内定义函数	C 不允许函数嵌套定义	函数定义写在所有函数之外
调用在前、定义在后且无原型	编译器不知道参数类型，按猜测处理	在文件开头写函数原型
`void f(int a) { return a; }`	`void` 函数不能返回值	改返回类型或去掉 `return` 值
`int f(int a, b)`	形参必须逐个指明类型	`int f(int a, int b)`
期望函数内改形参影响实参	值传递，形参只是副本	用指针或数组作参数
递归缺终止条件	无限递归，栈溢出	先写 base case
`static int n = 0;` 以为每次都重新置 0	静态局部变量只初始化一次	需要每次重置就用普通局部变量
头文件中定义函数体或变量	被多个文件包含时重复定义	头文件只放声明
`#include <mymath.h>`	尖括号只在系统目录查找	自定义头文件用双引号

第 8 章

数组和算法基础

8.1 一维数组的定义和初始化

数组（Array）是**相同类型**数据的有序集合，在内存中占据一段**连续**的存储单元。数组把”一批同类型数据”打包成一个整体，用一个名字统一管理，再用下标区分其中的每个元素，这正是后续排序、查找、字符串、矩阵运算的基础。

8.1.1 数组的定义

定义一维数组的一般形式为：

类型关键字　数组名[常量表达式];

例如 `int score[40];` 定义了一个名为 `score` 的整型数组，可存放 40 个 `int` 型数据。

语法规则有四点：

1. **长度必须是常量表达式**。标准 C（C89/C90）要求方括号内是编译期可求值的整型常量表达式，因此 `int n=40; int a[n];` 在 C89 下是错的；C99 起允许变长数组（VLA），但教材与多数评测环境仍按”常量”要求。
2. **下标从 0 开始**，合法下标范围是 $0 \sim n-1$。长度为 n 的数组，最后一个元素是 `a[n-1]`，`a[n]` 已经越界。
3. **数组名是这批存储单元的首地址**，是一个地址常量，不能被赋值、不能做自增自减：`a = b;` 与 `a++;` 都是非法表达式。
4. **编译器不做下标越界检查**。越界访问在语法上完全合法，运行时可能悄悄改掉别的变量，也可能直接崩溃——这是 C 语言最典型的”沉默错误”。

8.1.2 数组的初始化

数组可以在定义时初始化，常见形式见下表。

> **核心考点**　部分初始化时”补 0”只发生在**定义时的初始化**里；若是先定义 `int a[5];` 再在函数内逐个赋值，没赋到的元素仍是随机值。想把一个已存在的数组清零，应使用循环或 `memset`，不能写 `a = {0};`——数组名不可整体赋值。

表 8.1: 一维数组初始化形式对照

初始化写法	含义	未写出元素的值
`int a[5] = {1,2,3,4,5};`	全部元素依次赋初值	——
`int a[5] = {1,2,3};`	部分初始化	其余自动为 0
`int a[5] = {0};`	常见的” 整体清零” 写法	全部为 0
`int a[] = {1,2,3,4,5};`	省略长度，由初值个数决定	长度为 5
`int a[5];`	只定义不初始化	不确定（随机残留值）

8.1.3 内存模型与元素访问

`int score[40];` 在 32 位环境下的内存布局可以想成 40 个紧挨着的抽屉，每个抽屉 4 字节，总长 $40 \times 4 = 160$ 字节。第 i 个元素的首地址满足

$$\&\texttt{score}[i] = \&\texttt{score}[0] + i \times \texttt{sizeof(int)}$$

” 连续存放” 带来两个直接后果：一是可以用” 首地址 + 偏移量” 线性地走遍整个数组；二是下标运算的本质就是地址运算，这一点在第 11 章讲指针与数组的关系时会彻底展开。

遍历数组的标准写法是” 下标从 0 到 n-1 的 for 循环”：

```
int a[10], i, sum = 0;
for (i = 0; i < 10; i++) {   /* 注意是 < 10，不是 <= 10 */
    scanf("%d", &a[i]);
    sum += a[i];
}
printf("average = %.2f\n", (double)sum / 10);
```

8.2 二维数组的定义和初始化

8.2.1 二维数组的定义

定义形式为 类型数组名 `[行数][列数];`，例如 `int score[3][4];` 表示 3 行 4 列共 12 个 `int` 元素。元素用两个下标引用：`score[i][j]`，其中 $0 \le i \le 2$，$0 \le j \le 3$。

二维数组在内存中并不是” 二维” 的：C 语言按**行优先**（row-major）顺序把二维数组拉直成一维存放，先存第 0 行全部元素，再存第 1 行，依此类推。因此 `score[1][0]` 紧跟在 `score[0][3]` 之后。设列数为 N，则

$$\&\texttt{a}[i][j] = \texttt{a} + (i \times N + j) \times \texttt{sizeof(元素类型)}$$

表 8.2: `int a[3][4]` 的行优先存储布局

内存顺序	0	1	2	···	11
元素	`a[0][0]`	`a[0][1]`	`a[0][2]`	···	`a[2][3]`
偏移（元素个数）	0	1	2	···	11

8.2.2 二维数组的初始化

- **分行初始化**（推荐，可读性最好）：

```
int a[3][4] = {{1, 2, 3, 4},
               {5, 6, 7, 8},
               {9, 10, 11, 12}};
```

- **连续初始化**：`int a[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};`，编译器按行优先自动”折行”。
- **部分初始化**：`int a[3][4] = {{1},{5,6}};`，未写出的元素全部补 0。
- **省略行数**：`int a[][4] = {{1,2,3,4},{5,6,7,8}};` 合法；但**列数永远不能省略**，因为编译器要靠列数计算”一行有多长”才能确定行数。

8.2.3 二维数组的遍历

遍历二维数组要用双重循环，通常**外层控制行、内层控制列**：

```
#define M 3
#define N 4
int a[M][N], i, j;
for (i = 0; i < M; i++) {
    for (j = 0; j < N; j++)
        printf("%4d", a[i][j]);
    printf("\n");              /* 一行输出完再换行 */
}
```

> **核心考点** 若把两层循环写反（外层列、内层行），结果虽然也能”算对”，但访问顺序与内存顺序相反，缓存命中率变差；更常见的错误是把循环边界写成行数/列数互换，导致越界。养成”M 行 N 列、外 i 内 j”的固定习惯可以避免大半错误。

8.3 向函数传递一维数组

8.3.1 形参的三种等价写法

把一维数组传给函数时，下面三种形参写法完全等价：

```
void PrintArray(int a[10], int n);   /* 方括号里的 10 只是"提示", 不起约束作用 */
void PrintArray(int a[],   int n);
void PrintArray(int *a,    int n);   /* 编译器实际看到的就是这个 */
```

原因是：**数组名作实参时，传递的是首元素地址，不是整个数组**。C 语言没有”按值传递整个数组”这回事，形参数组在函数内部已经退化为指针，所以

- 方括号内写不写长度都一样，写了也不检查；
- 函数无法从形参得知数组有多少个元素，必须**另外传一个长度参数** n；
- 在函数内 `sizeof(a)` 得到的是**指针的大小**（4 或 8 字节），不是数组总字节数。

8.3.2 地址传递的效果

因为传的是地址，函数内对 `a[i]` 的修改会直接作用在实参数组上——这与普通变量”按值传递、函数内改了外面不变”的行为截然不同。

表 8.3: 普通变量与数组作函数参数的区别

	普通变量	一维数组
实参形式	`f(x)`	`f(a, n)`
传递内容	值的一份副本	首元素地址
函数内修改	不影响实参	直接影响实参元素
函数内 `sizeof`	变量本身的大小	指针的大小
需要额外传长度	不需要	必须传

```
/* 求数组元素最大值: 形参用 const 表明"只读", 是一种防御式编程习惯 */
int FindMax(const int a[], int n)
{
    int i, max = a[0];
    for (i = 1; i < n; i++)
        if (a[i] > max) max = a[i];
    return max;
}
```

8.4 排序和查找

在数组中搜索一个特定元素的过程称为**查找**（search），把数组元素按关键字重新排列的过程称为**排序**（sort）。被查找的值称为**查找键**（searchKey）。排序与查找是数组最重要的两类应用，也是”算法” 这一概念的入口。

8.4.1 交换法排序

思路：第 i 轮把 a[i] 与其后所有元素逐个比较，**一旦发现更大（或更小）就立即交换**。n 个元素共需 $n-1$ 轮。

```
void ExchangeSort(int a[], int n)       /* 由大到小 */
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (a[j] > a[i]) {          /* 边比较边交换，交换次数多 */
                temp = a[i]; a[i] = a[j]; a[j] = temp;
            }
}
```

特点：比较次数固定为 $n(n-1)/2$，但交换次数多，最坏情况下每比较一次都要搬三次数据。

8.4.2 冒泡法排序

思路：相邻两个元素比较，逆序就交换。每一轮结束后，当前未排序部分中的最值会” 冒” 到一端，因此下一轮可以少比一个。

```
void BubbleSort(int a[], int n)         /* 由小到大 */
{
    int i, j, temp, flag;
    for (i = 0; i < n - 1; i++) {
        flag = 0;
        for (j = 0; j < n - 1 - i; j++)   /* 已就位的尾部不再比较 */
            if (a[j] > a[j + 1]) {
                temp = a[j]; a[j] = a[j + 1]; a[j + 1] = temp;
                flag = 1;
            }
        if (flag == 0) break;             /* 本轮无交换，已经有序，提前结束
           */
    }
}
```

特点：最好情况（原本有序）只需 $n-1$ 次比较、0 次交换；加 flag 优化后可以提前退出。

8.4.3 选择法排序

思路：第 i 轮先在 a[i..n-1] 中**记下最值的下标**，本轮结束再交换一次。与交换法排序的区别只有一点：**交换发生在每轮末尾，而不是每次比较之后**。

```
void SelectionSort(int a[], int n)      /* 由小到大 */
{
    int i, j, k, temp;
    for (i = 0; i < n - 1; i++) {
        k = i;                           /* k 记录本轮最小元素的下标 */
        for (j = i + 1; j < n; j++)
            if (a[j] < a[k]) k = j;
        if (k != i) {                    /* 每轮至多交换一次 */
            temp = a[i]; a[i] = a[k]; a[k] = temp;
        }
    }
}
```

表 8.4: 三种简单排序算法对照（n 个元素）

算法	比较次数	交换次数	关键特征	稳定性
交换法排序	约 $n^2/2$	最坏约 $n^2/2$	每次比较都可能交换	不稳定
冒泡法排序	最坏 $n^2/2$	逆序对个数	相邻比较、最值逐步”冒”出	稳定
选择法排序	约 $n^2/2$	至多 $n-1$	先记下标、轮末交换一次	不稳定

8.4.4 线性查找（顺序查找）

思路最直白：从数组一端开始，用查找键 key 依次与每个元素比较，相等即返回下标；比完全部仍未相等，则返回 -1 表示”未找到”。

```
int LinSearch(const int a[], int n, int key)
{
    int i;
    for (i = 0; i < n; i++)
        if (a[i] == key) return i;    /* 找到，返回下标 */
    return -1;                        /* 全部比完仍未找到 */
}
```

特点：对数据**没有任何前提要求**（可以无序），实现最简单；代价是效率低——最坏要比较 n 次，平均约 $(n+1)/2$ 次，时间复杂度 $O(n)$。数据无序时，线性查找是唯一可行的办法。

8.4.5 折半查找

前提条件：数组元素已按关键字有序排列。在此前提下，折半查找（binary search，也称二分查找、对分搜索）每次比较都能排除一半元素。

算法步骤：

1. 设查找区间为 `[low, high]`，初值 `low = 0`，`high = n - 1`；
2. 取中间位置 `mid = (low + high) / 2`，把 `a[mid]` 与查找键 `key` 比较；
3. 若相等，查找成功，返回 `mid`；
4. 若 `key < a[mid]`（升序表），说明目标在前半段，令 `high = mid - 1`；否则令 `low = mid + 1`；
5. 重复 2–4，直到 `low > high`，此时区间为空，查找失败。

$$\texttt{mid} = \left\lfloor \frac{\texttt{low} + \texttt{high}}{2} \right\rfloor$$

```c
int BinarySearch(const int a[], int n, int key)
{
    int low = 0, high = n - 1, mid;
    while (low <= high) {               /* 区间非空才继续 */
        mid = (low + high) / 2;
        if (key == a[mid]) return mid; /* 找到，返回下标 */
        else if (key < a[mid]) high = mid - 1;
        else                   low  = mid + 1;
    }
    return -1;                          /* 未找到 */
}
```

效率：每比较一次就排除一半元素，因此最多所需的比较次数约为 $\log_2 n + 1$ 次。$n = 1024 = 2^{10}$ 时最坏只要 10 次左右比较，而顺序查找最坏要 1024 次。这就是”有序”换来的效率。

> **核心考点** 折半查找三处易错：（1）**忘记先排序**——对无序数组折半，结果无意义；（2）循环条件写成 `low < high`，会漏掉区间里只剩一个元素的情况；（3）区间收缩写成 `high = mid` 或 `low = mid`（没有 ±1），可能造成死循环。

8.5 向函数传递二维数组

二维数组作函数参数时，形参必须写成”数组的数组”，**第二维（列数）不能省略**：

```c
void PrintMatrix(int a[][4], int rows);      /* 正确：给出列数 4 */
void PrintMatrix(int (*a)[4], int rows);     /* 与上一行完全等价 */
```

表 8.5: 顺序查找与折半查找对照

算法	对数据的要求	基本思想	平均效率
顺序查找（线性查找）	无要求，可无序	从一端逐个与查找键比较	$O(n)$
折半查找（二分查找）	**必须有序**	每次与中间元素比较，排除一半	$O(\log_2 n)$

```
/* void PrintMatrix(int a[][], int rows);      错误：列数未知，无法计算地址 */
```

原因是：函数内部要用 `a[i][j]` 定位元素，必须先知道”一行有多少个元素”才能算出偏移量 $i \times N + j$；行数只影响循环边界，所以可以作为普通参数传入。

```
/* 求每行元素的平均值：rows 为实际行数，列数固定为 N */
#define N 4
void RowAverage(int a[][N], int rows)
{
    int i, j, sum;
    for (i = 0; i < rows; i++) {
        sum = 0;
        for (j = 0; j < N; j++) sum += a[i][j];
        printf("row %d average = %.2f\n", i, (double)sum / N);
    }
}
```

> **核心考点** 不要以为 `int **` 能接 `int a[3][4]`。二维数组是一片连续内存，`a` 的类型是”指向含 4 个 int 的数组”的指针，与”指向指针的指针”`int **` 是两种完全不同的类型；混用会在运行期按错误的地址取值（详见第 11 章）。

8.6 本章知识点小结

- 数组是**同类型、连续存储**的元素集合；下标从 0 起，合法范围 $0 \sim n-1$。
- 定义时长度须为常量表达式；初始化分全部、部分（自动补 0）、省略长度三种；数组名是地址常量，不能整体赋值、不能自增。
- 一维数组元素地址 `&a`$[i]$ = `&a`$[0] + i \times$ `sizeof(`元素`)`；二维数组按**行优先**拉直存放，`&a`$[i][j]$ = `a` $+ (iN + j) \times$ `sizeof(`元素`)`。
- 数组作函数实参传递的是首地址，形参 `a[]`、`a[10]`、`*a` 三者等价，必须另传长度；函数内 `sizeof` 失效。

- 二维数组作形参时列数不可省；行数用普通参数传入。
- 排序：交换法（边比边换）、冒泡法（相邻比较、可提前退出）、选择法（记下标、轮末交换）。查找：顺序查找无前提、$O(n)$；折半查找须有序、$O(\log_2 n)$。

8.7 本章常见错误小结

1. **用圆括号引用数组元素**：写成 `a(3)` 或 `a[2,3]`（把行列写进同一个方括号）。前者被当成函数调用，后者是逗号表达式的值 3，都不是" 第 3 个元素"。
2. **下标越界**：`for (i = 1; i <= n; i++) a[i] = 0;` 访问了 `a[n]`，编译器不报错，却已破坏相邻内存。
3. **给数组名赋值**：`a = b;`、`a++;`、" 数组整体赋初值" `a = {0};` 均非法；数组整体拷贝要逐元素或用 `memcpy`。
4. **函数内用 sizeof 求数组长度**：`int len = sizeof(a) / sizeof(a[0]);` 在形参 `a` 是数组名时可用，在函数内 `a` 已退化为指针，结果恒为 1 或 2，是经典陷阱。
5. **二维数组形参省掉列数**：`void f(int a[][])` 编译不过；即使写成 `int **a`，与二维数组也不是同一类型。
6. **对无序数组折半查找**：前提条件不成立，结果错误；应先排序。
7. **行列循环写反或边界用错**：遍历 `a[3][4]` 时用 `j < 3` 控制列，少访问了一列。

第9章

指针

9.1 变量的内存地址

程序中的每个变量都要占用一段存储单元，而内存是按**字节**编号的，每个字节有一个唯一的编号，这个编号就叫该字节的**地址**（address）。一个变量往往占若干个连续字节，C 语言规定：**变量所占存储单元的首字节地址就是这个变量的地址**。

取地址用单目运算符 `&`（取地址运算符，address-of operator）：

```
int a = 10;
printf("a = %d\n", a);
printf("&a = %p\n", (void *)&a);    /* %p 专门用于以十六进制输出地址 */
```

说明：

- `&` 的操作数必须是**内存中实实在在的对象**，如变量、数组元素、结构体成员。`&10`、`&(a+b)`、`&a++` 都非法，因为常量与表达式的中间结果不占（或不长期占）一个可寻址的存储单元。
- `&` 的结果是一个**地址值**，它的类型是” 指向操作数类型的指针”，`&a` 的类型是 `int *`。
- 输出地址必须用 `%p`（或 `%x` 加转换），用 `%d` 打印地址在 64 位环境下会截断，属常见错误。
- 同一个变量的地址在程序运行期间一般不变，但两次运行之间可能不同——**不要在程序里写死具体地址**。

表 9.1: 变量与地址的关系（示意，地址值仅为举例）

变量名	类型	存储单元（字节）	地址（首字节）
`char c`	1 字节	0x0061FE0F	0x0061FE0F
`int a`	4 字节	0x0061FE08–0x0061FE0B	0x0061FE08
`double d`	8 字节	0x0061FE00–0x0061FE07	0x0061FE00

9.2 指针变量的定义和初始化

9.2.1 指针变量的概念

地址本身也是一种数据，也可以存放在变量里。**专门用来存放变量地址的变量称为指针变量**（pointer variable）。指针变量与普通变量的区别只在于” 它肚子里装的是地址”，而不是普通的数值。

定义指针变量的一般形式为：

```
类型关键字  *指针变量名;
```

其中 * 是**定义的一部分**（声明符），不是运算符；前面的” 类型关键字” 称为该指针的**基类型**，表示” 这个指针指向什么类型的数据”。

```
int   *p;      /* p 是指向 int 型数据的指针，p 的类型是 int *      */
char  *q;      /* q 是指向 char 型数据的指针                       */
double *r;     /* r 是指向 double 型数据的指针                     */
int   *s, t;   /* 只有 s 是指针，t 是普通 int！这是极易错的一行     */
```

> **核心考点** `int *s, t;` 中 * 只作用于紧跟其后的 s。想让两个都是指针必须写 `int *s, *t;`。更稳妥的写法是” 一行为一个变量”：`int *s;` 另起一行 `int *t;`。

9.2.2 指针变量的初始化

指针变量可以用” 取某个变量的地址” 来初始化：

```
int a = 10;
int *p = &a;      /* 定义 p 的同时让它指向 a，推荐写法 */
int *q;
q = &a;           /* 先定义、后赋值，与上面等价 */
```

三点语法规则：

1. **基类型必须与被指向变量的类型一致**。`int a; char *p = &a;` 在语法上可能只给一条警告，但用 *p 按 1 字节解释 4 字节的 int，结果必然错。
2. **指针变量本身也占内存**，大小与平台有关：32 位环境通常 4 字节，64 位环境通常 8 字节，可用 `sizeof(int *)` 查证。所以” 指针占几个字节” 和” 它指向的数据占几个字节” 是两件不同的事。
3. **未初始化的指针是” 野指针”**，里面是随机的残留地址，可能指向任何地方；对它解引用等于在未知地址上读写，往往导致程序崩溃或数据被莫名改写。定义指针后若暂时没有目标，应初始化为 NULL：

```c
int *p = NULL;     /* NULL 是值为 0 的符号常量，表示"不指向任何有效单元" */
if (p != NULL) {   /* 使用前先判空，是一种防御式编程习惯 */
    /* ... */
}
```

9.2.3 指针的内存模型

把” 变量” 和” 指针变量” 放在一起看，内存里是这样的关系：

表 9.2: `int a = 10; int *p = &a;` 的内存模型

变量	地址	存储内容	说明
a	0x0061FE08	10	普通变量，存数值
p	0x0061FE00	0x0061FE08	指针变量，存的是 a 的地址

也就是说：p 的值是 &a，*p 访问到的是 a 这个存储单元，二者指向同一块内存。

9.3 间接寻址运算符

9.3.1 运算符 * 的含义

单目运算符 *（间接寻址运算符，也称解引用运算符，dereference operator）的作用是**通过指针访问它所指向的存储单元**：

```c
int a = 10, b;
int *p = &a;
b = *p;         /* 把 p 所指单元的值（即 a 的值 10）赋给 b   */
*p = 20;        /* 把 20 写入 p 所指单元，等价于 a = 20      */
printf("a = %d\n", a);   /* 输出 20：a 被"隔着指针"改掉了 */
```

核心认识：*p 与 a 表示**同一个存储单元**，所以 *p 既可以当右值读取，也可以当左值写入。

9.3.2 & 与 * 的关系

&（取地址）与 *（取内容）是一对互逆运算，满足

$$*(\&a) \equiv a, \qquad \&(*p) \equiv p$$

即” 先取地址再解引用” 得到原变量，” 先解引用再取地址” 得到原指针。

9.3.3 定义中的 * 与表达式中的 *

这是初学者最容易混淆的一点：

表 9.3: * 的两种身份

出现位置	身份	例子与含义
变量定义/声明中	类型的一部分（声明符）	`int *p;` 声明 p 是 `int *` 型变量，此处没有运算
表达式中	单目运算符	`b = *p;` 取出 p 所指单元的值

> **核心考点** `*p` 的类型是" 所指数据的类型"，p 的类型是" 指针类型"：对 `int *p` 来说，`*p` 是 `int`，p 是 `int *`。写错类型最常见的后果出现在 `printf` 里：`printf("%d", p)` 输出的是地址的低位，`printf("%d", *p)` 输出的才是数据。

9.3.4 指针的算术运算（预告）

指针可以做加减一个整数的运算，其含义是**按基类型大小移动**，而不是按字节移动：

$$p + i \equiv p \text{ 的地址值} + i \times \texttt{sizeof(基类型)}$$

例如 `int *p` 中 p+1 的实际地址值加 4 字节；`char *q` 中 q+1 只加 1 字节。指针还可以相减（得到相隔的元素个数）、比较大小（判断谁在前），但**两个指针相加没有意义**，编译器会直接报错。这一部分在第 11 章与数组结合后会讲得更透。

9.4 按值调用与模拟按引用调用

9.4.1 C 语言只有按值调用

C 语言中，函数调用时**实参的值被复制一份传给形参**，形参是函数内部的局部变量，函数里怎么改形参，都不会影响外面的实参——这叫**按值调用**（call by value）。

```
void SwapWrong(int x, int y)      /* 只交换了副本，实参不变 */
{
    int t = x; x = y; y = t;
}
/* 调用：SwapWrong(a, b);  a、b 的值原封不动 */
```

9.4.2 用指针模拟按引用调用

如果传给形参的不是变量的值，而是**变量的地址**，那么形参（指针）就指向了实参那块存储单元；函数里通过 * 形参写入，等于直接改写实参——效果上相当于其他语言的" 按引用调用"，所以称为**模拟按引用调用**（simulated call by reference）。

```
void Swap(int *px, int *py)      /* 形参是指针 */
{
    int t = *px;
    *px = *py;
    *py = t;
}
/* 调用时必须传地址：Swap(&a, &b); */
```

表 9.4: 按值调用与模拟按引用调用对照

	实参形式	形参形式	函数内能否改变实参
按值调用	`f(a)`	`int x`	不能，改的是副本
模拟按引用调用	`f(&a)`	`int *px`	能，`*px` 就是实参本身
数组作参数	`f(a, n)`	`int a[]`	能（数组名退化为地址，本质上仍是按值传递地址）

> **核心考点** 调用 Swap 时写成 Swap(a, b) 而不是 Swap(&a, &b)，编译器多半会给一条" 参数类型不匹配" 的警告或错误；若强行转换通过，函数会把 a、b 的**值**当地址去改写内存，运行期直接崩溃。凡是形参为指针的函数，调用时先问一句" 要不要加 &"。

9.5 用指针变量作函数参数的程序实例

指针作函数参数有三大典型用途：改写多个实参、让函数" 返回" 多个结果、在函数内操作大块数据（数组）而不必复制。

9.5.1 让函数返回多个结果

C 函数只能有一个返回值，需要带回多个结果时，用指针形参" 带出去"：

```
/* 同时求最大值和最小值：返回值给出状态，结果通过指针带回 */
void MaxMin(const int a[], int n, int *pmax, int *pmin)
{
    int i;
    *pmax = *pmin = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] > *pmax) *pmax = a[i];
```

```
        if (a[i] < *pmin) *pmin = a[i];
    }
}
/* 调用: MaxMin(a, 10, &max, &min); */
```

9.5.2 指针形参与数组形参的等价性

`int a[]` 与 `int *a` 作形参完全等价，因为数组名作实参传递的本来就是首地址；函数内既可以写 `a[i]`，也可以写 `*(a+i)`。这一点在第 11 章会系统展开。

9.5.3 常见写法与风格

- 只读取不修改时，形参加 `const`：`const int *p`，让编译器帮你拦住误写。
- 函数入口先检查指针是否为空：`if (p == NULL) return;`，这叫防御式编程。
- 形参是指针时，注释里写清” 输入/输出/输入输出”，避免调用者不知道该不该加 `&`。

9.6 函数指针及其应用

9.6.1 函数指针的定义与初始化

程序中的函数也要占用一段内存，**函数名就代表该函数代码的首地址**。因此可以定义指向函数的指针——**函数指针**：

```
int Max(int a, int b) { return a > b ? a : b; }

int (*pf)(int, int);      /* 定义函数指针 pf: 指向"两个 int 参数、返回 int"的
    函数 */
pf = Max;                 /* 函数名即地址, 等价于 pf = &Max;
                          */
int m = (*pf)(3, 5);      /* 通过指针调用, 等价于 pf(3, 5)
                           */
```

语法要点：

1. **括号不能省**。`int *pf(int, int);` 是” 返回 `int *` 的函数” 的声明，与函数指针是两回事。记忆口诀：`(*pf)` 先说明”pf 是指针”，再看外层。
2. 函数指针的**签名必须匹配**：参数个数、参数类型、返回类型都要一致，否则行为未定义。
3. 调用时 `(*pf)(a,b)` 与 `pf(a,b)` 等价。后者更简洁，但前者更能提醒读者” 这是通过指针调用”。
4. 函数指针可以指向不同函数，从而实现” 同一段代码、不同行为”。

9.6.2 函数指针作函数参数：回调

把函数指针作参数传入，被调函数在合适的时机” 回头调用” 它，这种机制称为**回调**（callback）。它是实现通用算法的标准手法：

```
/* 通用冒泡排序：比较规则由调用者以函数指针传入 */
void Sort(int a[], int n, int (*cmp)(int, int))
{
    int i, j, t;
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - 1 - i; j++)
            if (cmp(a[j], a[j + 1]) > 0) {   /* 用外部规则判断是否逆序 */
                t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
}
int Asc(int x, int y)  { return x - y; }     /* 升序规则 */
int Desc(int x, int y) { return y - x; }     /* 降序规则 */
/* 调用：Sort(a, n, Asc);  或  Sort(a, n, Desc); */
```

9.6.3 函数指针的典型应用场景

- **菜单驱动程序**：把” 加、减、乘、除” 四个函数放进函数指针数组，用下标选择，避免长长的 `switch`。
- **通用算法**：如上例的排序、查找、遍历框架，把” 比较/访问” 这一变化点抽出来。
- **回调与事件处理**：库函数在特定事件发生时调用用户注册的函数（标准库 `qsort` 的第四个参数就是比较函数指针）。

```
/* 函数指针数组：一个简易计算器 */
int Add(int a, int b) { return a + b; }
int Sub(int a, int b) { return a - b; }
int Mul(int a, int b) { return a * b; }

int (*op[3])(int, int) = { Add, Sub, Mul };   /* 三个元素，各指向一个函数 */
int r = op[1](7, 3);                          /* 调用 Sub，结果 4 */
```

> **核心考点** 不要返回**局部变量**的地址。函数返回后其栈帧被回收，返回的地址指向” 已不存在的变量”，这类指针叫悬垂指针（dangling pointer），用它是未定义行为。要返回指针，只能返回静态变量、动态分配的内存，或调用者传进来的缓冲区地址。

9.7 本章知识点小结

- 内存按字节编址，变量的地址是其首字节地址，用 `&` 取得，用 `%p` 输出。
- 指针变量存放地址，定义形式 类型 `*p;`，基类型决定”一次访问几个字节、如何解释这些字节”。
- 指针本身占 4 字节（32 位）或 8 字节（64 位）；未初始化即为野指针，应初始化为 `NULL` 并判空使用。
- `*` 是间接寻址运算符：`*p` 与 `p` 所指变量是同一存储单元；`&` 与 `*` 互逆。
- 定义中的 `*` 属于类型声明，表达式中的 `*` 是运算符，切勿混淆。
- 指针加减整数按”基类型大小”移动；两指针可相减、可比较，不可相加。
- C 只有按值调用；传地址（`&`）即可在函数内改写实参，称为模拟按引用调用。
- 函数名即函数代码首地址；函数指针定义须写 `(*pf)`，可用于回调、通用算法与函数指针数组。

9.8 本章常见错误小结

1. **使用未初始化的指针**：`int *p; *p = 5;` 把 5 写到随机地址，典型表现是”程序运行时崩溃”或”某个无关变量被改”。
2. **混淆 p 与 *p**：`scanf("%d", p);` 少了 `&`，`p` 若未初始化就是往随机地址写；应为 `scanf("%d", &a);` 或让 `p` 先指向 `a` 再传 `p`。
3. **指针类型不匹配**：`int *p; char *q = p;` 需要显式转换且语义可疑；`int *` 与 `double *` 混用会按错误的字节数取值。
4. **把 * 的优先级弄错**：`*p++` 是”先取 `*p`，再让 `p` 自增”；`(*p)++` 是”让 `p` 所指的值加 1”；`++*p` 与 `++(*p)` 同义。三者结果完全不同。
5. **试图交换指针本身**：在函数里写 `t = px; px = py; py = t;` 只交换了形参（局部指针），实参指针不受影响；要交换的是 `*px` 与 `*py`。
6. **返回局部变量的地址**：函数结束后该地址失效，返回的是悬垂指针。
7. **两个指针相加**：`p + q` 无意义，编译报错；只有相减（求元素个数差）才有定义。
8. **用%d 打印地址**：64 位环境下地址是 8 字节，`%d` 只取低 4 字节，输出错乱甚至崩溃。

第10章

字符串

10.1 字符串字面量

用一对双引号括起来的若干字符称为**字符串字面量**（string literal），也称字符串常量，例如 `"China"`、`"hello, world"`、`"a"`。

三条必须记牢的规则：

1. **编译器自动在末尾加一个结束符 `'\0'`**。`'\0'` 是转义字符，代表 ASCII 码为 0 的字符（空字符 NUL），它既不是字符 `'0'`（ASCII 48），也不是空格。
2. **字符串的存储长度 = 字符个数 + 1**。故 `"China"` 占 6 个字节，其中 5 个存字母、1 个存 `'\0'`；而 `strlen("China")` 的值是 5（长度不含结束符），`sizeof("China")` 的值是 6（含结束符）。
3. **字符串字面量存放在只读存储区**，试图修改它（如 `"China"[0] = 'c';`）是未定义行为，通常直接崩溃。

其他细节：

- 相邻的字符串字面量会被自动拼接：`"hello, " "world"` 等价于 `"hello, world"`，常用于把过长的字符串分行书写。
- 空串 `""` 是合法的，它只占 1 个字节（就放 `'\0'`），长度为 0。
- 字符串字面量的类型是”字符数组”，在表达式中使用时**退化为指向首字符的指针**（类型 `char *`），这正是 `printf("%s", "China")` 能工作的原因。

10.2 字符串的存储

C 语言没有专门的”字符串类型”，字符串是用**字符数组**来存储的：把每个字符依次放进数组元素，最后放一个 `'\0'` 作为结束标志。程序遍历时”看到 `'\0'` 就停”，因此 `'\0'` 是字符串的**边界**，没有它就只是一堆字符。

10.2.1 用字符数组存字符串

```
char s1[6] = "China";        /* 5 个字母 + '\0'，刚好放下，推荐 */
char s2[]  = "China";        /* 长度由初值自动定为 6              */
char s3[10] = "China";       /* 其余 4 个字节自动补 0             */
```

```
char s4[5] = "China";        /* 危险！没有位置放 '\0'，s4 不是字符串 */
char s5[]  = {'C','h','i','n','a'};   /* 只是字符数组，5 个元素，无 '\0' */
```

要点：

- 数组长度**至少**要是”字符数 + 1”。少给 1，`'\0'` 放不进去，后续按字符串处理时就会越过数组边界继续读，直到偶然遇到 0 才停。
- 用 `char s[] = "China";` 这类”双引号”形式初始化会自动补 `'\0'`；用逐个字符初始化（`char s5[] = {'C','h',...}`）则**不会**自动补，需要自己写 `'\0'` 或留出位置由”部分初始化补 0”的规则补上。
- 字符数组可以修改：`s1[0] = 'c';` 合法，因为数组有自己的存储副本。

10.3 字符指针

除字符数组外，还可以用**字符指针**指向一个字符串：

```
char *p = "China";        /* p 指向只读区中该字面量的首字符 'C' */
printf("%c\n", *p);       /* 输出 C   */
printf("%c\n", *(p+1));   /* 输出 h   */
p = "Hello";              /* 合法：改的是指针的指向，不是字符串内容 */
/* p[0] = 'c';            非法：试图修改只读区内容 */
```

表 10.1: `char s[] = "China";` 与 `char *p = "China";` 的区别

比较项	`char s[] = "China";`	`char *p = "China";`
本质	数组，自己拥有一片 6 字节空间	指针，只存一个地址
内容所在	栈（或全局区），可读写	只读存储区，不可写
能否修改字符	可以，`s[0]='c';`	不可以，未定义行为
能否改变指向	不可以，`s = ...` 非法	可以，`p = ...` 合法
`sizeof`	6（整个数组的字节数）	4 或 8（指针本身的大小）
`strlen`	5	5

> **核心考点** ”数组名不能赋值、指针可以改指向”是理解这两者的关键：`s` 是地址常量，`p` 是存放地址的变量。需要修改字符串内容时用字符数组；只是”引用”一段现成文本、不打算改它时，用 `const char *p` 更省空间也更安全。

10.4 字符串的访问和输入/输出

10.4.1 如何访问字符串中的单个字符

字符数组和字符指针都可以用下标访问，也可以用指针运算访问，两种写法完全等价：

$$s[i] \equiv *(s+i)$$

```c
char s[] = "China";
int i = 0;
while (s[i] != '\0') {          /* 以结束符为循环条件，而不是固定长度 */
    putchar(s[i]);
    i++;
}
/* 等价的指针写法 */
char *p = s;
while (*p != '\0') { putchar(*p); p++; }
```

注意循环条件应写” 当前字符不是 '\0'”，这样函数对任意长度的字符串都成立，不必额外传长度。

10.4.2 字符串的输入/输出

表 10.2: 常用字符串输入/输出函数

函数	所属头文件	要点
`printf("%s", s)`	stdio.h	从 s 起逐字符输出，遇 '\0' 停
`puts(s)`	stdio.h	输出字符串并**自动换行**，不能格式化
`scanf("%s", s)`	stdio.h	跳过前导空白，**遇空白即停**，不检查数组边界
`gets(s)`	stdio.h	读一行（可含空格），**不检查边界**，已从 C11 标准中移除
`fgets(s, n, stdin)`	stdio.h	最多读 $n-1$ 个字符，**安全**，但会保留换行符
`getchar/putchar`	stdio.h	单个字符的输入/输出，常用于逐字符处理

```c
char name[20];
scanf("%s", name);              /* 数组名本身就是地址，不要再加 & */
scanf("%19s", name);            /* 更安全：限定最多读 19 个字符 */
/* 读入含空格的一整行，推荐写法 */
if (fgets(name, sizeof(name), stdin) != NULL) {
    size_t len = strlen(name);
    if (len > 0 && name[len - 1] == '\n')   /* 去掉 fgets 留下的换行符 */
        name[len - 1] = '\0';
}
```

> **核心考点** `scanf("%s", s)` 不检查目标数组有多大，输入超长就会**越界写入**（缓冲区溢出）；`gets` 同样危险且已被标准移除。工程上应使用 `fgets` 或带宽度限制的 `scanf("%19s", s)`。另外，`scanf` 读字符串时形参已经是地址，写成 `&name` 虽常能"碰巧工作"，但类型不对，属坏习惯。

10.5 字符串处理函数

标准库 `string.h` 提供了一组字符串处理函数，它们都以 `'\0'` 作为结束标志，因此都要求传入的字符数组**必须以 `'\0'` 结尾**。

表 10.3: 常用字符串处理函数

函数原型	功能	说明与风险
`size_t strlen(const char *s)`	求长度	不含 `'\0'`；返回值类型为 `size_t`（无符号），与 `int` 比较时注意符号问题
`char *strcpy(char *d, const char *s)`	复制	连 `'\0'` 一起复制；**不检查 d 的容量**
`char *strncpy(char *d, const char *s, size_t n)`	限长复制	最多复制 n 个字符；若 s 短于 n，其余补 0
`char *strcat(char *d, const char *s)`	连接	把 s 接到 d 末尾；d 必须有足够剩余空间
`char *strncat(char *d, const char *s, size_t n)`	限长连接	最多接 n 个字符，并自动补 `'\0'`
`int strcmp(const char *a, const char *b)`	比较	按 ASCII 码逐字符比；<0、$=0$、>0 分别表示小于、相等、大于，**不要依赖具体返回值是 -1 或 1**
`int strncmp(const char *a, const char *b, size_t n)`	限长比较	只比前 n 个字符

理解这些函数的最好办法是自己实现一遍：

```
/* 求字符串长度：走到 '\0' 为止 */
size_t MyStrlen(const char *s)
{
    size_t n = 0;
    while (*s != '\0') { n++; s++; }
    return n;
}
/* 复制字符串：连 '\0' 一起搬 */
```

```c
char *MyStrcpy(char *d, const char *s)
{
    char *p = d;
    while ((*d++ = *s++) != '\0')
        ;                       /* 空循环体：赋值表达式的值就是刚复制的字符 */
    return p;
}
/* 比较字符串：返回首个不同字符的差值 */
int MyStrcmp(const char *a, const char *b)
{
    while (*a != '\0' && *a == *b) { a++; b++; }
    return (unsigned char)*a - (unsigned char)*b;
}
```

> **核心考点** 字符串不能用 = 赋值、不能用 == 比较内容。`s1 = s2;` 是给数组名赋值（非法）；`if (s1 == s2)` 比较的是两个**地址**，几乎永远为假。正确做法是 `strcpy` 与 `strcmp`。

10.6 向函数传递字符串

字符串作函数参数时，传的仍然是**首字符的地址**，形参写作 `char *` 或 `char s[]`，二者等价，而且**不需要另传长度**——长度信息由 `'\0'` 自带。

```c
/* 统计字符串中某字符出现的次数：形参加 const 表示只读 */
int CountChar(const char *s, char ch)
{
    int count = 0;
    for (; *s != '\0'; s++)
        if (*s == ch) count++;
    return count;
}
/* 调用：int c = CountChar("banana", 'a');   结果为 3 */
```

若函数要**修改**字符串内容，形参不能加 `const`，且实参必须是可写的字符数组（不能是字符串字面量）：

```c
/* 删除字符串中所有指定字符：原地操作，"读写指针"法 */
void DeleteChar(char *s, char ch)
{
    char *p = s, *q = s;      /* p 读、q 写 */
    while (*p != '\0') {
        if (*p != ch) *q++ = *p;
        p++;
    }
```

```
    *q = '\0';
}
```

10.7 从函数返回字符串指针

函数可以返回 `char *`，但**绝不能返回函数内部局部数组的地址**——函数返回后栈帧销毁，那个地址随即失效（悬垂指针）。合法的三种做法：

1. **返回传入的缓冲区**（最常用）：结果写进调用者提供的数组，再把它的地址返回。

```
char *ToUpperCopy(char *dst, const char *src)
{
    char *p = dst;
    while (*src != '\0') {
        *p++ = (*src >= 'a' && *src <= 'z') ? (char)(*src - 32) : *src;
        src++;
    }
    *p = '\0';
    return dst;                 /* dst 由调用者提供，函数返回后仍有效 */
}
```

2. **返回静态数组**的地址：`static char buf[64]`；存放在静态存储区，函数结束后仍存在；缺点是所有调用共享同一块缓冲，下一次调用会覆盖上次结果。
3. **返回动态分配的内存**：用 `malloc` 在堆上申请，由调用者负责 `free`（见第 11 章）。

> **核心考点** 判据很简单：函数返回的地址所指向的内存，在函数返回后**是否还活着**？栈上的局部数组”死了”，静态区与堆上的”还活着”，调用者传进来的”由调用者管”。答错这一问，就写出悬垂指针。

10.8 本章扩充内容

10.8.1 const 类型限定符

`const` 表示”只读”，加在指针上时位置不同、含义完全不同：

函数形参用 `const char *s` 有两层好处：一是向调用者承诺”我不改你的数据”，二是编译器会拦住函数内不小心的写操作。`strlen`、`strcmp` 等标准函数的原型都是这样写的。

表 10.4: const 与指针的三种组合

声明	谁不能改	记忆方法
`const char *p`	所指内容	const 在 * 左边，管” 内容”
`char * const p`	指针本身	const 在 * 右边，管” 指针”
`const char * const p`	两者都不能改	两边都有 const

10.8.2 字符处理函数

ctype.h 提供了一批按字符分类与转换的函数，参数是 int（通常传 (unsigned char)c），返回非 0 表示” 真”：

表 10.5: 常用字符处理函数（ctype.h）

函数	判断/作用	函数	判断/作用
isalpha	是否为字母	isdigit	是否为数字字符
isalnum	是否为字母或数字	isspace	是否为空白字符
isupper	是否为大写字母	islower	是否为小写字母
toupper	转成大写	tolower	转成小写

自己写” 判断是否大写” 时容易写成 `c >= 'A' && c <= 'Z'`（可行但不可移植）；用 `isupper(c)` 更清晰，也不依赖 ASCII 的具体编码。

10.8.3 数值字符串向数值的转换

```c
#include <stdlib.h>
int    i = atoi("123");         /* 字符串 -> int    */
long   l = atol("123456");      /* 字符串 -> long   */
double d = atof("3.14");        /* 字符串 -> double */
/* 更稳健：strtol 能通过第二个参数报告"停在哪"，便于发现非法输入 */
char *end;
long v = strtol("123abc", &end, 10);   /* v = 123, end 指向 'a' */
/* 反向：数值 -> 字符串 */
char buf[32];
sprintf(buf, "%d-%.2f", 7, 3.5);       /* buf 里得到 "7-3.50" */
```

要点：atoi 遇到非数字字符就停下，**不报错**，非法串返回 0，无法与真正的 0 区分；需要错误检测时用 strtol。反向转换时 sprintf 不检查目标缓冲区大小，安全版本是 snprintf。

10.9 本章知识点小结

- 字符串字面量用双引号表示，编译器自动在末尾加 `'\0'`；存储字节数 = 字符数 + 1；字面量在只读区，不可修改。
- 字符串用字符数组存储，数组长度至少为” 字符数 + 1”；逐字符初始化不会自动补 `'\0'`。
- 字符指针 `char *p = "China";` 只存首字符地址，改指向合法、改内容非法；字符数组则相反。
- `s[i]` 与 `*(s+i)` 等价；遍历以 `'\0'` 为结束条件。
- 输入：`scanf("%s")` 遇空白停且不查边界；`gets` 不安全；`fgets` 安全但要处理换行符。输出：`printf("%s")`、`puts`。
- 字符串函数 `strlen/strcpy/strcat/strcmp` 及其 `n` 版本都以 `'\0'` 为前提；`strcpy` 不检查容量。
- 字符串不能用 `=` 赋值、不能用 `==` 比较内容，须用 `strcpy` 与 `strcmp`。
- 传字符串给函数只传地址，长度由 `'\0'` 自带；返回字符串指针时不能返回局部数组地址。
- `const` 位置决定” 锁内容” 还是” 锁指针”；字符分类用 `ctype.h`，数值转换用 `atoi/strtol` 与 `sprintf`。

10.10 本章常见错误小结

1. **数组长度少给 1**：`char s[5] = "China";` 放不下 `'\0'`，后续 `strlen` 会越界读，结果随机。
2. **逐字符初始化后当字符串用**：写 `char s[] = {'C','h','i','n','a'};` 这样逐个字符初始化，数组里就没有 `'\0'`，再用 `printf("%s", s)` 输出，程序会一直读到内存里偶然出现的 0 才停。
3. **修改字符串字面量**：`char *p = "China"; p[0] = 'c';` 往只读区写，通常直接崩溃。
4. **用 = 赋值、用 == 比较**：`s1 = s2;` 非法；`if (s1 == s2)` 比的是地址，不是内容。
5. **scanf 读字符串加 &**：`scanf("%s", &s)` 类型不对；数组名本身就是地址。
6. **scanf(”%s”) 读含空格的一行**：遇空格即停，只能读到第一个单词，应改用 `fgets`。
7. **strcpy 目标空间不足**：`char d[5]; strcpy(d, "China");` 溢出，是缓冲区溢出的经典来源。
8. **返回局部数组的地址**：函数返回后指针悬垂，读到的可能是任意垃圾值。
9. **忘记 #include <string.h>**：在 C99 及以后，隐式声明被禁止，调用 `strlen` 会直接编译失败。

第 11 章

指针和数组

11.1 指针和一维数组间的关系

11.1.1 数组名就是首元素地址

一维数组在内存中连续存放，数组名本身就是这段内存的**首元素地址**：

```
int a[5] = {10, 20, 30, 40, 50};
int *p = a;           /* 等价于 p = &a[0]; */
printf("%d\n", *p);         /* 10 */
printf("%d\n", *(p + 2));   /* 30，与 a[2] 相同 */
```

由此得到本课程最重要的一组等价关系：

$$a[i] \equiv *(a+i), \qquad \&a[i] \equiv a+i$$

也就是说：**下标运算的本质是"首地址 + 偏移，再解引用"**。正因为加法可交换，`*(a+i)` 与 `*(i+a)` 相同，于是 `a[i]` 与 `i[a]` 在语法上也等价——但后者可读性极差，只作理解，不要写进代码。

11.1.2 数组名的两个"特殊"之处

- **数组名是地址常量**，不能赋值、不能自增：`a = p;`、`a++;` 都非法。指针变量 `p` 则可以随意改变指向（`p++`），这是数组名与指针最本质的区别。
- **`a` 与 `&a` 的值相同，类型不同**。`a` 的类型是 `int *`（指向一个 `int`），`&a` 的类型是 `int (*)[5]`（指向"含 5 个 int 的数组"，即整个数组）。因此 `a+1` 只移动 4 字节，而 `&a+1` 移动 20 字节。

表 11.1: 一维数组的三种指针写法（`int a[5]`）

表达式	类型	含义	等价写法
`a`	`int *`	首元素地址	`&a[0]`
`a+i`	`int *`	第 i 个元素的地址	`&a[i]`
`*(a+i)`	`int`	第 i 个元素的值	`a[i]`
`&a`	`int (*)[5]`	整个数组的地址	——

11.1.3 用指针遍历数组

```
int a[5] = {1, 2, 3, 4, 5}, *p, sum = 0;
for (p = a; p < a + 5; p++)       /* 指针作循环变量 */
    sum += *p;
/* 等价的指针相减：两个指针相减得到相隔的元素个数 */
int n = (int)(p - a);             /* 循环结束后 p == a + 5，故 n = 5 */
```

指针相减的两个前提：两个指针指向同一数组（或数组末尾的下一个位置），且类型相同；结果是以" 元素个数" 为单位的 `ptrdiff_t` 型整数，不是字节差。

> **核心考点** 不要以为" 指针法比下标法快"。对 `a[i]` 与 `*(a+i)`，编译器生成的机器码通常完全相同；可读性才是选择标准。真正需要指针的场合是：函数要改变实参、要在函数间传递" 一段数据"、要实现动态数据结构。

11.2 指针和二维数组间的关系

11.2.1 二维数组名的类型是" 行指针"

设 `int a[3][4];`。二维数组可以看作" 由 3 个一维数组组成的数组"，每个一维数组含 4 个 `int`。因此：

- `a` 的类型是 `int (*)[4]`——**指向" 含 4 个 int 的数组" 的指针**，俗称**行指针**（数组指针）。`a+1` 跳过一整行，地址值增加 $4 \times 4 = 16$ 字节。
- `a+i` 指向第 i 行（这一整行），`*(a+i)` 退化后是第 i 行首元素的地址，类型为 `int *`。
- 再偏移 j 个元素：`*(a+i)+j` 就是 `&a[i][j]`；解引用得 `*(*(a+i)+j)`，即 `a[i][j]`。

$$a[i][j] \equiv *(a[i]+j) \equiv *(*(a+i)+j)$$

```
int a[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
int (*p)[4] = a;                  /* 行指针：一次跳一行 */
printf("%d\n", *(*(p + 1) + 2));    /* 7，即 a[1][2] */
printf("%d\n", p[2][3]);            /* 12：行指针也能用两维下标 */
```

11.2.2 数组指针与指针数组

这两个名字极易混，区别只在" 先结合谁"：

口诀：**看括号**。`(*p)` 说明"p 先是指针"，其余是它指向的东西；`*q[4]` 中 `[]` 优先级高于 `*`，所以 `q` 先是数组。

表 11.2: `int (*p)[4]` 与 `int *q[4]` 的区别

声明	读法	本质	一次偏移
`int (*p)[4];`	p 是指针，指向含 4 个 int 的数组	**数组指针**（行指针），一个指针	一整行（16 字节）
`int *q[4];`	q 是数组，含 4 个元素，元素是 int *	**指针数组**，四个指针	一个指针大小

11.2.3 二维数组作函数参数

形参必须写成”行指针”形式，第二维（列数）不可省：

```
void PrintMatrix(int (*a)[4], int rows)   /* 与 int a[][4] 完全等价 */
{
    int i, j;
    for (i = 0; i < rows; i++) {
        for (j = 0; j < 4; j++) printf("%4d", a[i][j]);
        printf("\n");
    }
}
```

为什么不能写成 `int **a`？因为 `a[i][j]` 会被编译成 `*(*(a+i)+j)`：第一步 `a+i` 若按 `int **` 解释，只前进一个指针大小，取出的”行地址”其实是元素数据本身，再解引用就完全错了。`int **` 对应的是”指针数组 + 每行单独分配”的结构，与二维数组不是一回事（见第四节）。

> **核心考点** 二维数组的”行”概念是编译期信息：编译器靠列数 N 算出 `a+i` 的步长。所以列数一旦省略，地址运算无从谈起。传二维数组时，把行数当普通参数传，把列数写进形参类型，是最稳的写法。

11.3 指针数组及其应用

11.3.1 指针数组用于表示多个字符串

表示多个字符串有两种做法：二维字符数组，或”指针数组 + 各字符串”。后者往往更灵活、更省空间。

```
/* 做法一：二维字符数组，每行定长 10，浪费空间，行不可整体交换 */
char names1[3][10] = {"China", "Japan", "Korea"};

/* 做法二：指针数组，三个指针各指向一个字符串字面量 */
char *names2[3] = {"China", "Japan", "Korea"};

/* 排序时只交换指针，不必搬运字符串内容 */
```

```
8 char *t = names2[0]; names2[0] = names2[2]; names2[2] = t;
```

表 11.3: 二维字符数组与字符指针数组对照

比较项	`char a[3][10]`	`char *p[3]`
内存布局	30 字节连续空间，每行定长	3 个指针 + 各字符串本身
空间利用	短串也按最长者占位，浪费	按实际长度分配，节省
能否修改内容	能（自己拥有副本）	不能（指向只读区字面量）
交换两”行”	必须 `strcpy` 搬内容	交换指针即可，$O(1)$
排序效率	低（大量字节搬运）	高（只搬 4/8 字节的地址）

11.3.2 指针数组用于表示命令行参数

带参数运行程序时，操作系统把命令行按空白拆成若干字符串，交给 `main`：

```
1 /* argc: 参数个数（含程序名本身）; argv: 指针数组，各元素指向一个参数字符串
     */
2 int main(int argc, char *argv[])
3 {
4     int i;
5     printf("program: %s\n", argv[0]);        /* argv[0] 是程序名 */
6     for (i = 1; i < argc; i++)
7         printf("arg %d: %s\n", i, argv[i]);
8     return 0;                                  /* argv[argc] 恒为 NULL，可作结
          束标志 */
9 }
```

形参 `char *argv[]` 与 `char **argv` 完全等价——这正是”指针数组名退化为指向指针的指针”的实例。若程序名是 `copy`，运行 `copy a.txt b.txt`，则 `argc` 为 3，`argv[1]` 是 `"a.txt"`、`argv[2]` 是 `"b.txt"`。

11.4 动态数组

11.4.1 C 程序的内存映像

一个运行中的 C 程序，内存大致分成四块：

”局部数组开不大”就是因为它在栈上：数组一大会栈溢出；”数组长度必须编译期确定”也是因为栈帧大小在编译时就要算好。要按运行期才知道的长度开数组，只能上堆。

表 11.4: C 程序的内存分区

区域	存放内容	生命周期与特点
代码区	机器指令、字符串字面量等只读数据	程序运行期间始终存在，只读
静态存储区	全局变量、`static` 变量	程序开始到结束；未初始化则自动为 0
栈（stack）	函数的局部变量、形参、返回地址	随函数调用自动分配、返回自动回收；容量小、速度快
堆（heap）	`malloc/calloc/realloc` 申请的内存	由程序员手动申请、手动 `free`；容量大、速度较慢

11.4.2 动态内存分配函数

表 11.5: 四个动态内存管理函数（stdlib.h）

函数原型	功能与要点
`void *malloc(size_t size)`	申请 `size` 字节，**不初始化**；成功返回首地址，失败返回 `NULL`
`void *calloc(size_t n, size_t size)`	申请 $n \times$ `size` 字节并**全部清零**
`void *realloc(void *p, size_t size)`	把已申请块调整为 `size` 字节；可能搬家，**必须用返回值重新赋值**
`void free(void *p)`	释放 `p` 指向的堆内存；`p` 必须是 `malloc` 系列返回的地址，且只能释放一次

三条铁律：

1. `malloc` 返回 `void *`，赋给具体类型指针时需强制转换：`(int *)malloc(...)`。
2. **必须检查返回值是否为 `NULL`**（内存不足时会失败），否则后续解引用会崩溃。
3. **有申请必有释放**，且释放后应把指针置 `NULL`，防止误用（`free(p); p = NULL;`）。

11.4.3 长度可变的一维动态数组

```
int n, i, *a;
printf("n = ");
scanf("%d", &n);                          /* 长度由运行期决定 */
a = (int *)malloc(n * sizeof(int));       /* 申请 n 个 int 的空间 */
if (a == NULL) {                          /* 防御式编程：先判空 */
    printf("Not enough memory!\n");
    return -1;
}
for (i = 0; i < n; i++) a[i] = i * i;     /* 用法与普通数组完全相同 */
free(a);                                  /* 用完必须释放 */
a = NULL;                                 /* 置空，避免悬垂 */
```

11.4.4 长度可变的二维动态数组

两种常见建法，各有取舍：

```
/* 方式一：指针数组 + 每行单独申请（行与行之间不保证连续，但可让各行列数不同） */
int rows = 3, cols = 4, i, j, **a;
a = (int **)malloc(rows * sizeof(int *));
if (a == NULL) return -1;
for (i = 0; i < rows; i++) {
    a[i] = (int *)malloc(cols * sizeof(int));
    if (a[i] == NULL) return -1;
}
a[1][2] = 7;
for (i = 0; i < rows; i++) free(a[i]);   /* 释放顺序：先各行，再指针数组 */
free(a);

/* 方式二：一次性申请连续块，再按行切分（内存连续，缓存友好） */
int *buf = (int *)malloc(rows * cols * sizeof(int));
int \textbf{b  = (int })malloc(rows * sizeof(int *));
for (i = 0; i < rows; i++) b[i] = buf + i * cols;
/* 释放：free(b); free(buf);  顺序不能反，否则 buf 地址丢失 */
```

> **核心考点** 释放顺序与申请顺序**相反**，并且”每块都要释放一次、只释放一次”。方式一漏掉 `free(a[i])` 会造成内存泄漏；方式二先 `free(buf)` 再 `free(b)` 虽不报错，但 `b[i]` 已成悬垂。工程上把”申请/释放”成对写在一个函数的两端，能避免绝大多数疏漏。

11.5 本章扩充内容

11.5.1 常见的内存错误及其对策

11.5.2 缓冲区溢出攻击

原理：栈上分配局部数组时，数组之后紧跟的是函数的返回地址、保存的寄存器等控制信息。若程序对输入不做长度检查（如 `strcpy`、`gets`、`scanf("%s")` 写入定长数组），超长输入会**越过数组边界**，把精心构造的数据写到返回地址上；函数返回时便”跳到”攻击者指定的代码处执行。这类攻击称为**缓冲区溢出攻击**（buffer overflow），是历史上最著名的安全漏洞类型之一。

典型隐患写法：

```
char buf[10];
strcpy(buf, input);         /* input 超过 9 个字符即溢出 */
gets(buf);                  /* 完全不检查长度，已被标准移除 */
scanf("%s", buf);           /* 同理，危险 */
```

表 11.6: 动态内存的六类典型错误

错误	表现与后果	对策
申请后不判 NULL	内存不足时解引用空指针，立即崩溃	每次都写 `if (p == NULL)`
越界读写	破坏堆管理信息或相邻数据，崩溃点与出错点相距很远	下标严格 $< n$；用 calloc 便于发现未初始化
重复 free	破坏空闲链表，常见"double free" 崩溃	释放后置 NULL
free 非堆地址	释放栈变量或数组名，未定义行为	只释放 malloc 系列返回的地址
释放后继续使用	悬垂指针，读到已被复用的数据	释放后不再引用该指针
忘记释放	内存泄漏，长时间运行的程序逐渐耗尽内存	申请与释放成对出现，路径分支都要覆盖

防御手段（从代码到系统层层设防）：

- **代码层**：改用 fgets、strncpy、snprintf、scanf("%9s")，并显式检查长度；对来自外部的输入一律视为" 不可信"。
- **编译器层**：开启栈保护（GCC 的 -fstack-protector）插入" 金丝雀" 值，返回前校验其是否被改写。
- **系统层**：栈不可执行（NX）、地址随机化（ASLR）等机制，使溢出即使发生也难以被利用。

11.6 本章知识点小结

- 数组名是首元素地址、是地址常量；a[i] ≡ *(a+i)，&a[i] ≡ a+i。
- a 与 &a 值同类型异：前者步长一个元素，后者步长整个数组。
- 指针可遍历数组、可相减求元素个数；相减须同数组、同类型。
- 二维数组名是行指针 int (*)[N]：a+i 跳一行，*(a+i) 是行首地址；
- 元素访问满足 a[i][j] ≡ *(*(a+i)+j)，也等于 *(a[i]+j)。
- 数组指针 int (*p)[4] 是一个指针；指针数组 int *q[4] 是四个指针。
- 二维数组作形参，列数不可省，行数另行传入；int ** 不能接二维数组。
- 指针数组可表示多个字符串（省空间、可交换指针排序）与命令行参数 argv。
- C 程序内存分代码区、静态区、栈、堆；动态数组用 malloc/calloc/realloc 在堆上申请，用 free 释放。
- 动态内存的五条纪律：判空、不越界、不重复释放、释放后置空、申请与释放成对。

11.7 本章常见错误小结

1. **给数组名赋值或自增**：`a = p;`、`a++;` 编译报错，因为 `a` 是地址常量。
2. **混淆 `a+1` 与 `&a+1`**：前者移动一个元素，后者移动整个数组；在二维数组里则分别对应” 跳一个元素” 与” 跳一行”。
3. **把二维数组传给 `int **`**：类型不匹配，运行期按错误的地址取值，是最隐蔽的错误之一。
4. **函数内用 `sizeof(a)` 求数组长度**：形参已退化为指针，结果恒为 4 或 8。
5. **`malloc` 不判空、不释放**：前者在内存紧张时崩溃，后者造成泄漏。
6. **释放顺序错误或重复释放**：二维动态数组要” 先各行、后指针数组”；`free` 两次是未定义行为。
7. **释放后继续使用**：`free(p)` 之后又读 `*p`，读到的是已被复用的内存。
8. **把 `realloc` 的返回值丢掉**：`realloc` 可能搬家，写成 `realloc(p, n);` 而不接收返回值，旧指针可能已失效。
9. **对字符串字面量做 `strcpy` 的目标**：字面量在只读区，写入即崩溃。

第12章

结构体和数据结构基础

12.1 从基本数据类型到抽象数据类型

前面用过的类型（`int`、`double`、`char`、数组）都只能描述”单个值”或”同类型的一批值”。而现实中的对象往往是**多种属性的组合**：一个学生有学号（整数）、姓名（字符串）、成绩（实数），这三者逻辑上属于同一个整体，却类型不同。用三个互不相干的变量表示，既容易在函数间传递时遗漏，也无法表达”它们是一伙的”这层语义。

把”数据”和”作用于数据的操作”打包在一起，只暴露必要的接口、隐藏内部表示，这种思想叫**抽象数据类型**（Abstract Data Type，ADT）。C 语言没有类的语法，但用**结构体**可以把若干不同类型的数据成员组织成一个整体，再用一组函数充当”操作”，就能近似地实现 ADT：

- **数据封装**：结构体把相关数据成员捆在一起，作为一个整体定义、传递、返回。
- **操作分离**：对结构体的操作由一组函数完成（如 `PrintStudent`、`SortByScore`），函数原型就是”接口”。
- **信息隐藏**：调用者只需要知道结构体”有什么”，不必关心成员在内存里如何排列。

这正是本章的主线：从基本类型 → 结构体 → 结构体数组 → 结构体指针 → 链表等动态数据结构。

12.2 结构体的定义

12.2.1 为什么要定义结构体类型

结构体（structure）是由若干**类型可以不同**的数据成员（member）组成的构造类型。它解决的第一个问题是”命名”：给”学生”这种复合数据起一个类型名，之后就能像 `int` 一样用它定义变量、作函数参数、放进数组。解决的第二个问题是”整体性”：可以整体赋值、整体传递，而不必逐个成员搬运。

12.2.2 结构体类型的定义与变量定义

定义结构体类型的一般形式：

```
struct 结构体名 {
    类型1  成员名1;
    类型2  成员名2;
```

```
    ...
};
```

```
struct student {            /* 定义"学生"类型：此时不分配内存，只是一个模板 */
    long   num;             /* 学号 */
    char   name[20];        /* 姓名 */
    double score;           /* 成绩 */
};                          /* 末尾的分号不能少！ */
```

定义变量有三种写法：

```
/* 写法一：先定义类型，再定义变量（最常用，类型可复用） */
struct student s1, s2;

/* 写法二：定义类型的同时定义变量 */
struct student { long num; char name[20]; double score; } s1, s2;

/* 写法三：匿名结构体，只此一次，无法再用该类型定义新变量 */
struct { long num; char name[20]; double score; } s3;
```

> **核心考点** `struct student { ... };` 这一行末尾的分号必须写。漏掉它，编译器会认为你在继续声明变量，错误信息往往指向下一行，看起来毫无关系——这是结构体最经典的”报错位置与错误位置不一致”陷阱。

12.2.3 用 typedef 定义数据类型

`typedef` 给已有类型起别名，能让结构体的使用摆脱 `struct` 关键字：

```
typedef struct student {
    long   num;
    char   name[20];
    double score;
} Student;                  /* Student 是"struct student"的别名 */

Student s1;                 /* 不必再写 struct */
```

两种常见习惯：一是如上的”`typedef struct 标签 {...} 别名;`”，标签与别名并存；二是匿名加别名 `typedef struct {...} Student;`，此后只能用别名。别名本身也是类型名，可以再 `typedef` 出指针类型：`typedef Student *StudentPtr;`。

12.2.4 结构体变量的初始化

```
Student s1 = {20240101, "Zhang San", 92.5};    /* 按成员顺序依次赋值 */
Student s2 = {20240102, "Li Si"};              /* 部分初始化：score 得 0.0 */
Student s3 = {.name = "Wang Wu", .num = 20240103, .score = 88.0};
                                               /* C99 指定初始化，顺序无关 */
```

若成员本身是数组或结构体，则用嵌套的花括号：`{20240101, "Zhang San", {1990, 5, 20}}`。

12.2.5 嵌套的结构体

成员可以是另一个结构体类型（**不能是自身**，但可以是指向自身的指针，链表正是这样构造的）：

```
struct date { int year, month, day; };

struct employee {
    long  id;
    char  name[20];
    struct date birthday;      /* 嵌套：成员是一个结构体 */
    struct date hireDay;
};
/* 引用最内层成员要一层层点下去 */
e.birthday.year = 1990;
```

12.2.6 结构体变量的引用

- **成员运算符 .**：`s1.num`、`s1.name`、`s1.score`。成员可以像普通变量一样参与运算、赋值、输入输出。
- **同类型结构体可以整体赋值**：`s1 = s2;` 会把所有成员逐字节复制过去（含数组）。
- **但整体比较、整体输入输出都不允许**：`s1 == s2` 编译错误；`printf("%s", s1)` 也编译错误。要比较就逐成员比较，要输出就逐成员输出。
- 成员是数组时，**不能对成员数组名整体赋值**：`s1.name = "Li Si";` 非法，须用 `strcpy` 函数逐个字符复制。

12.2.7 结构体所占内存的字节数

结构体的字节数**不等于**各成员字节数的简单相加，编译器会插入” 填充字节” 以满足**内存对齐**（alignment）要求：每个成员的起始偏移必须是其自身大小的整数倍，结构体总大小必须是” 最大成员大小” 的整数倍。

```
struct A { char c; int i; };   /* 成员字节和为 5，实际 sizeof 通常为 8 */
struct B { int i; char c; };   /* 实际 sizeof 通常也为 8 */
```

```
struct C { char c; double d; };/* 通常为 16: 1 + 7 填充 + 8 */
```

表 12.1: `struct A { char c; int i; };` 的典型内存布局（32 位）

偏移	0	1–3	4–7
内容	`c`	填充（未使用）	`i`

实用结论：求结构体大小一律用 `sizeof`，不要自己加；需要把结构体整体写进文件或通过网络发送时，填充字节里可能是随机值，应使用 `memset` 清零或用 `fwrite` 逐成员写出。

12.3 结构体数组的定义和初始化

12.3.1 结构体数组的定义

```
Student stu[3];                 /* 3 个学生，每个都含完整的成员 */
#define N 40
Student class1[N];              /* 一片连续内存，元素是结构体 */
```

结构体数组的元素在内存中依次连续排列，因此可以用下标访问、可以整体传给函数（退化为指针），与普通数组别无二致。

12.3.2 结构体数组的初始化

```
Student stu[3] = {
    {20240101, "Zhang San", 92.5},
    {20240102, "Li Si",     85.0},
    {20240103, "Wang Wu",   78.5}
};                              /* 外层花括号是数组，内层是每个结构体 */
```

遍历与排序都很自然：把”按成绩排序”写成”交换结构体元素”，成员顺序跟着整体走，不会出现”学号换了成绩没换”的错位。

```
/* 按成绩由高到低排序：元素整体交换，成员自动保持一致 */
void SortByScore(Student a[], int n)
{
    int i, j; Student t;
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (a[j].score > a[i].score) { t = a[i]; a[i] = a[j]; a[j] = t;
               }
}
```

12.4 结构体指针的定义和初始化

12.4.1 指向结构体变量的指针

```
Student s1 = {20240101, "Zhang San", 92.5};
Student *p = &s1;              /* p 指向一个结构体变量 */
```

通过指针访问成员有两种等价写法：

$$(*p).name \equiv p \to name$$

-> 称为**指向运算符**，它的左边必须是指针。二者等价的原因很直白：”先解引用拿到结构体，再取成员”与”顺着指针直接取成员”是同一件事。

> **核心考点** 为什么必须写 (*p).name 而不能写 *p.name？因为成员运算符 . 的优先级高于 *，*p.name 会被解释成 *(p.name)，而 p 是指针、没有 name 成员，直接编译错误。所以要么加括号，要么用 ->。

12.4.2 指向结构体数组的指针

```
Student stu[3] = {...};
Student *p = stu;              /* 指向第 0 个元素 */
p++;                           /* 移动一个"结构体"的大小，指向 stu[1] */
printf("%s\n", p->name);       /* 输出第 1 个学生的姓名 */
printf("%s\n", (p + 1)->name); /* 第 2 个学生 */
```

指针的加减仍以”元素”为单位，只不过这里的”一个元素”是一整个结构体（其大小由 sizeof 给出），这正是指针运算”按基类型移动”规则的又一次体现。

12.5 向函数传递结构体

结构体可以按值传递，也可以传地址，二者差别很大：

表 12.2: 结构体的两种传参方式

方式	形参写法	特点
值传递	`void f(Student s)`	复制整个结构体；函数内修改不影响实参；成员多、数组大时开销大
地址传递	`void f(Student *p)`	只复制一个地址；函数内可通过 p->x 改实参；效率高，推荐
只读地址传递	`void f(const Student *p)`	传地址但禁止修改，兼具效率与安全

```c
/* 只读访问：传地址 + const，既快又安全 */
void PrintStudent(const Student *p)
{
    printf("%-10ld %-20s %6.1f\n", p->num, p->name, p->score);
}
/* 需要修改实参时传地址 */
void AddBonus(Student *p, double delta) { p->score += delta; }
/* 调用 */
PrintStudent(&stu[0]);
AddBonus(&stu[1], 5.0);        /* stu[1].score 真的被改了 */
```

经验法则：结构体较大或需要在函数内修改时传指针；结构体很小（如只有两三个 `int`）且不需修改时，值传递也无可厚非。

12.6 共用体

共用体（union，也称联合体）的语法与结构体几乎一样，但内存模型完全相反：**所有成员共用同一段存储空间**，起始地址相同。

```c
union data {
    int   i;
    char  ch;
    double d;
};
union data u;
u.i = 65;                  /* 此刻按 int 解释这 8 个字节 */
printf("%c\n", u.ch);      /* 再按 char 解释同一段内存的首字节 */
```

表 12.3: 结构体与共用体的对比

比较项	结构体 struct	共用体 union
内存分配	各成员各占一块，依次排列	所有成员共用一块，起始地址相同
占用字节数	各成员之和 + 填充	最大成员的大小（并按对齐要求取整）
同时有效	所有成员同时有效	同一时刻只有一个成员有效
典型用途	描述” 由多个属性组成的对象”	节省空间；同一段内存的多种解释

要点：共用体的值取决于**最后一次写入**的是哪个成员；读一个” 不是最后写入” 的成员，得到的只是那段字节的另一种解释（结果依赖具体实现），不是类型转换。共用体最实用的两个场景是节省存储（同一字段在不同情形下类型不同）与探测字节序（写入 `int` 1 后看首字节是否为 1）。

12.7 枚举数据类型

枚举（enumeration）用于表示”取值只有有限几个”的变量，如星期、颜色、状态：

```
enum weekday { SUN, MON, TUE, WED, THU, FRI, SAT };
enum weekday today;
today = WED;                    /* 直接用名字，可读性远好于写 3 */
```

规则：

- 枚举元素是**整型常量**，默认从 0 开始依次加 1（SUN=0，SAT=6）；可以显式指定 `enum {A = 1, B, C = 10, D};`，则 B=2、D=11。
- 枚举变量的本质是整型，可以参与比较、可以用于 `switch`，但不能直接 `scanf("%d", &today)`（类型不匹配，需先读入 `int` 再转换）。
- 枚举的价值在**可读性与可维护性**：用 `if (state == RUNNING)` 远比 `if (state == 2)` 清楚；元素增删时也不必改遍全程序的魔数。
- C 语言对枚举**不做严格类型检查**（`today = 100;` 可能只给警告），这是它与 C++ 的差别之一。

12.8 动态数据结构——单向链表

12.8.1 问题的提出

数组有两个先天限制：**长度固定**（编译期确定，运行中不能伸缩）、**插入删除代价高**（要在中间插入一个元素，后面所有元素都得后移）。若数据量在运行中不断变化，就需要一种”按需申请、用指针串起来”的结构——**链表**（linked list）。

12.8.2 链表的定义

链表由若干**结点**（node）组成，每个结点含两部分：**数据域**（存数据）与**指针域**（存下一个结点的地址）。结点之间靠指针相连，内存上**不必连续**。

```
typedef struct node {
    int          data;      /* 数据域 */
    struct node *next;      /* 指针域：指向同类型结点，形成"自引用" */
} Node;
```

- **自引用**必须写 `struct node *next`，不能写 `Node *next`——因为在 `typedef` 完成之前，`Node` 这个名字还不存在。
- 通常用一个**头指针** `Node *head` 记住第一个结点；最后一个结点的 `next` 置为 `NULL`，表示链表结束。

- 空链表就是 `head == NULL`。

表 12.4: 数组与链表对照

比较项	数组	单向链表
内存布局	连续	离散，靠指针相连
长度	编译期固定	运行期按需增长
随机访问	$O(1)$，`a[i]` 直接定位	$O(n)$，必须从头走
插入/删除	$O(n)$，需移动元素	$O(1)$（已定位前驱时），只改指针
额外开销	无	每个结点多一个指针

12.8.3 单向链表的建立

建表就是” 反复申请结点、接到链尾（或链头）”：

```
/* 尾插法：新结点接在表尾，输出顺序与输入顺序一致 */
Node *CreateList(void)
{
    Node *head = NULL, *tail = NULL, *p;
    int x;
    while (scanf("%d", &x) == 1 && x != -1) {   /* 以 -1 表示输入结束 */
        p = (Node *)malloc(sizeof(Node));
        if (p == NULL) return head;             /* 申请失败，返回已有链表 */
        p->data = x;
        p->next = NULL;
        if (head == NULL) head = p;             /* 第一个结点 */
        else              tail->next = p;       /* 接到表尾 */
        tail = p;                               /* 更新尾指针 */
    }
    return head;
}
/* 遍历：从头走到 NULL */
void PrintList(Node *head)
{
    Node *p;
    for (p = head; p != NULL; p = p->next)
        printf("%d ", p->data);
    printf("\n");
}
```

若改用**头插法**（`p->next = head; head = p;`），则每次插在表头，输出顺序与输入顺序**相反**——这也是” 逆序建表” 的常用技巧。

12.8.4 单向链表的删除操作

删除的关键是**找到前驱**：让前驱”绕过”待删结点。删除头结点要单独处理（因为没有前驱），这也是很多实现引入”带头结点（哨兵）”链表的原因。

```
/* 删除第一个值为 key 的结点，返回新的头指针 */
Node *DeleteNode(Node *head, int key)
{
    Node *p = head, *pre = NULL;
    while (p != NULL && p->data != key) {   /* 找目标结点及其前驱 */
        pre = p;
        p = p->next;
    }
    if (p == NULL) return head;             /* 没找到 */
    if (pre == NULL) head = p->next;        /* 删的是头结点 */
    else             pre->next = p->next;   /* 让前驱绕过 p */
    free(p);                                /* 释放被删结点 */
    return head;
}
```

12.8.5 单向链表的插入操作

在结点 p 之后插入新结点 s，两条语句的**顺序不能颠倒**：

```
s->next = p->next;     /* 第一步：新结点先接上后半个链 */
p->next = s;           /* 第二步：前驱再指向新结点     */
```

若先写 `p->next = s;`，`p->next` 原来的值（后半条链的入口）就丢失了，`s->next` 再也接不回去，链表从 p 处**断链**——这是链表操作最经典的错误。

> **核心考点** 链表的所有操作都可以归结为三句话：**先接后、再断前**（插入）；**找到前驱才能删除**；**每次 malloc 都要有对应的 free**。写链表代码时先在纸上画出 3 个结点的图，指针改一条划一条，能避免绝大多数断链与内存泄漏。

12.9 本章扩充内容

12.9.1 栈和队列

- **栈**（stack）：只允许在**一端**（栈顶）插入和删除，特点是**后进先出**（LIFO）。基本操作是入栈 push 与出栈 pop。可用数组实现（顺序栈，需判栈满/栈空），也可用链表实现（链栈）。函数调用、表达式求值、括号匹配都依赖栈。

- **队列**（queue）：一端入队（队尾）、另一端出队（队首），特点是**先进先出**（FIFO）。用数组实现时为避免” 假溢出”，通常做成**循环队列**，用 `(rear+1) % MAX` 回绕；用链表实现则只需维护队首、队尾两个指针。

12.9.2 树和图

- **树**（tree）：一种**层次**结构，有唯一的根结点，每个结点可有若干子结点且只有一个父结点（根除外）。最常用的是**二叉树**（每个结点至多两个子结点），其遍历分先序、中序、后序与层次遍历。文件目录、表达式结构、查找树都是树的实例。
- **图**（graph）：由顶点集合与边集合构成，结点之间可以任意相连，可带权、可有向。社交网络、地图路径、任务依赖都是图。

12.9.3 数据的逻辑结构和存储结构

表 12.5: 逻辑结构与存储结构

分类	类型	说明
逻辑结构	集合、线性结构、树形结构、图状结构	数据元素之间的逻辑关系，与内存无关
存储结构	顺序存储、链式存储、索引存储、散列存储	逻辑结构在内存中的实现方式

同一个逻辑结构可以有多种存储实现：线性表既可以用数组（顺序存储，随机访问快），也可以用链表（链式存储，插入删除快）。” 选哪种” 取决于实际最频繁的操作是什么，这正是数据结构设计的核心权衡。

12.10 本章知识点小结

- 结构体把不同类型的数据成员组织成一个整体，是 C 语言实现抽象数据类型的基础。
- 结构体类型定义末尾的**分号不能少**；定义类型不分配内存，定义变量才分配。
- `typedef` 可为结构体起别名，之后无需再写 `struct`。
- 成员用 `.` 访问；同类型结构体可整体赋值，但不可整体比较、整体输入输出。
- 结构体大小受**内存对齐**影响，一律用 `sizeof` 求，不要手工相加。
- 结构体数组的元素连续存放，可整体传给函数；排序时整体交换元素，成员不会错位。
- 结构体指针用 `->` 访问成员：`(*p).name` ≡ `p->name`；`p++` 移动一个结构体。
- 传结构体：小结构可值传递，大结构或需修改时传指针，只读时加 `const`。
- 共用体所有成员共享同一段内存，大小取最大成员；枚举用命名常量提高可读性。

- 链表结点含数据域与指针域，靠 `struct node *next` 自引用；插入”先接后、再断前”，删除要先找前驱。

12.11 第十一节：本章常见错误小结

1. **结构体定义漏分号**：`struct student { ... }` 后面少写 `;`，报错却指向下一行。
2. **忘记写 struct 或别名**：定义了 `struct student`，却直接写 `student s;`，除非有 `typedef`，否则编译错误。
3. **用 == 比较结构体**：`if (s1 == s2)` 非法；应逐成员比较，或比较 `memcmp`（注意填充字节）。
4. **直接 scanf 整个结构体**：`scanf("%s", s1)` 非法，必须逐个成员读。
5. **给成员数组整体赋值**：`s1.name = "Li Si";` 非法，须用 `strcpy` 函数复制。
6. **用 *p.name 访问成员**：优先级错误，应为 `(*p).name` 或 `p->name`。
7. **自己累加结构体大小**：忽略填充字节，导致 `fread/fwrite` 读写错位。
8. **链表插入顺序颠倒**：先 `p->next = s;` 导致后半条链丢失，链表断链。
9. **删除结点忘记 free**：结点从链上摘下来了，内存却没还，形成泄漏。
10. **自引用写成别名**：`typedef struct node { Node *next; } Node;` 在 `typedef` 完成前 `Node` 尚不存在，编译失败。
11. **返回局部结点地址**：函数内 `Node n;` 的地址在函数返回后失效，应 `malloc`。

第13章

文件操作

13.1 二进制文件和文本文件

此前程序的数据都存放在变量里，程序一结束就全部消失。要把数据**长期保存**，必须写进外存上的**文件**（file）——存放在外部介质上、有名字的一组相关数据。C 语言把文件看作**字节流**，不关心它在磁盘上如何组织，程序只负责” 按顺序读/写字节”。

按数据的组织方式，文件分两类：

表 13.1: 文本文件与二进制文件

比较项	文本文件（ASCII 文件）	二进制文件
存储形式	每个字符按编码存成字节	数据在内存中的字节原样写入
例：整数 1234	存 4 个字符 `'1''2''3''4'`，4 字节	存 4 字节的补码
可读性	可直接用记事本查看、便于交换	需专用程序解析，通常不可读
空间与速度	占空间多、需转换，较慢	占空间少、无需转换，较快
精度	浮点数转文本可能损失精度	完全保真
换行处理	文本模式下 `\n` 在 Windows 上被写成 2 字节	不做任何转换

要点：**同一个数，在文本文件和二进制文件里的字节数可能完全不同**。” 文件里有多少字节” 与” 有多少个数据” 是两件事，这正是第 13.5 节 `fread/fwrite` 用” 块数” 作返回值的原因。

13.2 文件的打开和关闭

13.2.1 文件指针

C 程序通过**文件指针**操作文件。`FILE` 是系统在 `stdio.h` 中定义的结构体类型，里面记录了缓冲区地址、当前读写位置、错误标志等信息：

```
#include <stdio.h>
FILE *fp;                    /* 定义文件指针变量，此时尚未与任何文件关联 */
```

程序启动时，系统自动打开三个标准流：`stdin`（标准输入，键盘）、`stdout`（标准输出，屏幕）、`stderr`（标准错误）。此前用的 `printf`、`scanf` 其实都在操作 `stdout`、`stdin`。

13.2.2 打开文件：fopen

```
FILE *fopen(const char *filename, const char *mode);
```

打开成功返回文件指针，**失败返回 NULL**。使用模式（mode）见下表。

表 13.2: fopen 的常用打开模式

模式	含义	文件不存在时
`"r"`	只读，文本方式	**打开失败**，返回 NULL
`"w"`	只写，文本方式	创建新文件；**已存在则清空**
`"a"`	追加写，文本方式	创建新文件；已有内容保留，写在末尾
`"r+"`	读写，文本方式	打开失败
`"w+"`	读写，文本方式	创建；已存在则清空
`"a+"`	读写 + 追加，文本方式	创建；写总在末尾
`"rb"/"wb"/"ab"`	同上，**二进制**方式	同对应的文本模式

```
FILE *fp = fopen("score.txt", "r");
if (fp == NULL) {                       /* 必须判空，这是最容易漏的一步 */
    printf("Cannot open file!\n");
    return -1;
}
```

13.2.3 关闭文件：fclose

```
int fclose(FILE *fp);       /* 成功返回 0，失败返回 EOF */
```

关闭做三件事：把缓冲区里尚未写出的数据**真正写进文件**、释放缓冲区、断开指针与文件的联系。因此**写完不关闭，数据可能还在缓冲区里没有落盘**。

> **核心考点** 两条纪律：（1）fopen 之后一定判 NULL，否则后续操作全部作用在空指针上，程序立刻崩溃；（2）fopen 与 fclose 必须成对，函数每一条返回路径都要关闭文件，否则数据丢失、句柄泄漏。特别注意 `"w"` 模式：文件已存在时会被**立刻清空**，误用一次就丢数据。

13.3 按字符读写文件

最基本的读写单位是字符，适用于文本文件的复制、统计、逐字符处理：

```
int fgetc(FILE *fp);            /* 成功返回读到的字符（int 型），失败或到末
    尾返回 EOF */
```

```
int fputc(int ch, FILE *fp);     /* 成功返回写入的字符，失败返回 EOF
                  */
```

```
/* 文件复制：读一个、写一个，直到 EOF */
FILE *in = fopen("in.txt", "r"), *out = fopen("out.txt", "w");
int ch;
if (in == NULL || out == NULL) { printf("open error\n"); return -1; }
while ((ch = fgetc(in)) != EOF)      /* 用返回值判结束，最可靠 */
    fputc(ch, out);
fclose(in);
fclose(out);
```

关于 EOF 与 feof：

- `EOF` 是 `stdio.h` 中定义的符号常量，值为 -1，表示" 读取失败或已到文件末尾"。
- `feof(fp)` 返回非 0 表示" 上一次读操作因为到达末尾而失败"。注意它**是" 读完之后" 才知道的**，不能在读之前用它判断" 还有没有内容"。
- 因此正确写法是 `while ((ch = fgetc(fp)) != EOF)`；写成 `while (!feof(fp)) { ch = fgetc(fp); ... }` 会**多处理一次**末尾的伪数据——这是文件操作最经典的错误。

13.4 按格式读写文件

`fprintf` 与 `fscanf` 的用法与 `printf`、`scanf` 几乎一样，只是多了一个文件指针参数，适合**文本文件**里按" 人类可读的格式" 存取：

```
int fprintf(FILE *fp, const char *format, ...);
int fscanf (FILE *fp, const char *format, ...);
```

```
/* 写：每个学生一行，字段用空格分隔 */
fprintf(fp, "%ld %s %.1f\n", s.num, s.name, s.score);
/* 读：返回值是"成功读入的项数"，用它判断数据是否完整 */
int n = fscanf(fp, "%ld %s %lf", &s.num, s.name, &s.score);
if (n != 3) { /* 数据不完整或格式不符，应停止读取 */ }
```

要点：

- `fscanf` 的**返回值**是成功赋值的项数。循环读文件时，用" 读到的项数是否等于预期" 作结束条件，比用 `feof` 更稳。
- `fscanf` 读 `%s` 时遇空白即停，因此**字符串里不能有空格**；姓名含空格时应改用 `fgets` 整行读入再解析。

- fprintf 与 fscanf 的格式必须严格对应，否则数据错位且不易察觉。文本文件适合调试与数据交换，不适合存浮点数组（有精度与体积损失）。

13.5 按数据块读写文件

fread、fwrite 以”块”为单位读写，一次搬若干个固定大小的字节，是**二进制文件**的标准做法，也最适合结构体数组：

```
size_t fread (void *ptr, size_t size, size_t n, FILE *fp);   /* 读 n 块，每块 size 字节 */
size_t fwrite(const void *ptr, size_t size, size_t n, FILE *fp);
```

返回值是”成功读/写的块数”，不是字节数；正常时应等于 n，小于 n 说明出错或已到末尾。

```
Student stu[3] = {{20240101,"Zhang San",92.5},
                  {20240102,"Li Si",85.0},
                  {20240103,"Wang Wu",78.5}};

/* 写：把整个数组一次性写进二进制文件 */
FILE *fp = fopen("stu.dat", "wb");
if (fp == NULL) return -1;
size_t w = fwrite(stu, sizeof(Student), 3, fp);   /* 3 块，每块一个结构体 */
fclose(fp);
if (w != 3) printf("write error\n");

/* 读：读到数组里，用返回值判断实际读了几条 */
fp = fopen("stu.dat", "rb");
if (fp == NULL) return -1;
Student buf[10];
size_t r = fread(buf, sizeof(Student), 10, fp);   /* 最多读 10 条 */
fclose(fp);
printf("read %u records\n", (unsigned)r);
```

> **核心考点** 用 fread/fwrite 存结构体时，写进去的是**内存里的字节，包括对齐填充字节**。所以：（1）读写必须用同一个结构体定义，否则字段错位；（2）填充字节里是随机值，文件里会带”垃圾”，需要干净的文件时先 memset 清零；（3）这种文件不可跨编译器/平台直接通用。

13.6 本章扩充内容

13.6.1 文件的随机读写

前面都是" 从头顺序读写"。文件指针内部记录着" 当前读写位置"，用 fseek 可以把它挪到任意位置，从而实现随机访问：

```
int  fseek(FILE *fp, long offset, int whence);   /* 成功返回 0，失败返回非 0 */
long ftell(FILE *fp);                            /* 返回当前读写位置（距文件首的字节数） */
void rewind(FILE *fp);                           /* 回到文件开头，等价于 fseek(fp,0,SEEK_SET) */
```

表 13.3: fseek 的基准位置

基准常量	含义	常见用法
SEEK_SET	从文件开头算起	fseek(fp, 0, SEEK_SET) 回到开头
SEEK_CUR	从当前位置算起	fseek(fp, -4, SEEK_CUR) 后退 4 字节
SEEK_END	从文件末尾算起	fseek(fp, -8, SEEK_END) 定位到倒数第 8 字节

随机读写最典型的用法是" 按记录号直接定位"，不必把前面所有记录读一遍：

```
/* 定位到第 k 条记录（k 从 0 开始），读取一条 */
long offset = (long)k * sizeof(Student);
if (fseek(fp, offset, SEEK_SET) == 0)
    fread(&s, sizeof(Student), 1, fp);
/* 文件里共有多少条记录？ */
fseek(fp, 0, SEEK_END);
long total = ftell(fp);
printf("records = %ld\n", total / (long)sizeof(Student));
```

注意：fseek 之后必须**清掉流上的 EOF 标志**（clearerr）才能继续正常读写；在文本文件中按字节定位是不可移植的（换行可能占 2 字节），随机访问应使用二进制文件。

13.6.2 标准输入/输出重定向

freopen 可以把一个已经打开的标准流" 改接到" 文件上，从而让 printf/scanf 直接读写文件，代码一行不必改：

```
freopen("input.txt",  "r", stdin);    /* 此后的 scanf 从文件读 */
freopen("output.txt", "w", stdout);   /* 此后的 printf 写进文件 */
/* 调试时想把输出恢复到屏幕，可再 freopen("CON", "w", stdout)（Windows） */
```

在命令行下也可以不改程序直接重定向（由操作系统完成同样的工作）：

表 13.4: 命令行的输入/输出重定向

写法	作用	说明
`prog > out.txt`	标准输出重定向到文件	**覆盖**写入
`prog >> out.txt`	标准输出追加到文件	保留原内容
`prog < in.txt`	标准输入重定向自文件	交互输入改为读文件

13.7 本章知识点小结

- C 把文件看作字节流；文本文件按字符编码存储、可读但占空间，二进制文件按内存字节存储、紧凑保真但不可读。
- 文件通过 `FILE *` 操作：`fopen` 打开（失败返回 `NULL`）、`fclose` 关闭（把缓冲区落盘）。
- 打开模式 `r/w/a` 及带 `+`、带 `b` 的组合含义要记清，尤其 `"w"` 会清空已有文件。
- 按字符：`fgetc/fputc`；用 `EOF` 判断结束，不要用 `feof` 作循环条件。
- 按格式：`fprintf/fscanf`，适合文本文件；`fscanf` 的返回值是成功读入的项数。
- 按数据块：`fread/fwrite`，适合二进制文件与结构体数组；返回值是**块数**。
- 随机读写：`fseek/ftell/rewind`，基准为 `SEEK_SET/CUR/END`，按记录号定位是常见用法。
- 重定向：`freopen` 改接标准流，命令行用 `>`、`>>`、`<`。

13.8 本章常见错误小结

1. **不检查 fopen 的返回值**：文件不存在时 `fp` 为 `NULL`，后续 `fgetc(fp)` 直接崩溃。
2. **忘记 fclose**：写出的数据可能还留在缓冲区，程序异常结束时文件内容残缺；也会泄漏文件句柄。
3. **误用 `"w"` 模式**：本想追加却用了 `"w"`，原文件被清空，且无法恢复。
4. **用 `feof` 作 while 条件**：`while (!feof(fp))` 会在末尾多读一次，多输出一条”伪记录”。
5. **用 `%s` 读带空格的内容**：`fscanf` 遇空白即停，只能读到第一个单词。
6. **忽略 fscanf 的返回值**：数据格式不符时读到一半就失败，若继续使用变量，用的是未更新的旧值。
7. **把 fread 的返回值当字节数**：它是**块数**；要算字节数应乘 `size`。
8. **文本模式打开二进制文件**（或反之）：Windows 下会做换行转换或把 `0x1A` 当文件结束，数据被破坏。

9. **随机读写时忘清 EOF 标志**：一次读到末尾后，`fseek` 到别处再读仍会立刻返回 `EOF`，需 `clearerr(fp)`。
10. **路径里的反斜杠没转义**：C 字符串中 \ 是转义引导符，写 Windows 路径必须写成两个反斜杠，或直接用正斜杠。

第 14 章

简单的游戏设计

14.1 动画的基本原理

电影与动画都建立在**视觉暂留**之上：画面以足够快的速度连续替换（一般每秒 24 帧以上），人眼就会把一帧帧静止图像看成连续运动。程序里的动画因此归结为一个循环——**帧循环**（game loop）：

1. **处理输入**：读取玩家这一帧按了什么键（不阻塞）；
2. **更新状态**：按规则改变坐标、速度、分数等数据；
3. **清屏并重绘**：把新的状态画到屏幕上；
4. **延时**：控制帧率，让画面速度稳定、别把 CPU 占满；
5. 回到第 1 步。

关键认识：**屏幕上看到的一切都只是”数据的一份画法”**。游戏的本质是”维护一组状态变量 + 每帧按规则更新它们 + 每帧把它们画出来”，图形库只负责最后那一步。

14.1.1 几个必要的手段

表 14.1: 控制台动画的常用手段

手段	典型做法	作用与注意点
清屏	`system("cls")`（Windows）	每帧前清屏；频繁调用会闪烁，可用光标定位代替
光标定位	把光标移到指定行列再输出	只重画变化的部分，比整体清屏更平滑
延时	`Sleep(ms)`（windows.h）、`delay`	**单位是毫秒**；延时太短画面过快、CPU 占用高
非阻塞按键	`kbhit()` + `getch()`（conio.h）	有键才读，无键立即返回，不会卡住帧循环
双缓冲	先在内存里画好整帧再一次性显示	消除”边画边显示”造成的闪烁

> **核心考点** 帧循环里最容易犯的错是”用阻塞式输入”。若直接写 `getchar()`，程序会停在原地等按键，画面就不再更新。必须用”有键则读、无键继续”的非阻塞方式，这是游戏程序与普通批处理程序在结构上的根本区别。

14.2 迷宫游戏

14.2.1 地图的数据表示

迷宫的本质是一张**二维字符数组**：每个格子用一个字符表示其性质。

```
#define ROWS 15
#define COLS 25
char map[ROWS][COLS] = {
    "#########################",
    "#S          #           #",
    "# #### #### # #### #### #",
    "#    #    #   #    #    #",
    "#### # ## ##### ## # ####",
    "#    #  #     #  #      #",
    "# ####  #######  #### ###",
    "#      #       #       E#",
    "#########################"
};
```

约定：`'#'` 为墙、空格为通路、`'S'` 为起点、`'E'` 为出口。把地图写成**字符串数组**比手写 `map[i][j] = '#'` 更直观，也便于修改。

14.2.2 状态与主循环

除地图外，还需要几个状态变量：玩家当前行列 (pr, pc)、是否胜利的标志。主循环为：

```
int pr = 1, pc = 1;          /* 玩家位置（起点） */
int running = 1;
while (running) {
    Render(map, pr, pc);      /* 画地图和玩家 */
    int key = ReadKey();      /* 非阻塞读一个按键 */
    int nr = pr, nc = pc;     /* 先假定不动 */
    switch (key) {
        case 'w': nr--; break;            /* 上 */
        case 's': nr++; break;            /* 下 */
        case 'a': nc--; break;            /* 左 */
        case 'd': nc++; break;            /* 右 */
        case 'q': running = 0; break;     /* 退出 */
    }
```

```
    /* 边界检查 + 撞墙判断：只有目标格不是墙才真正移动 */
    if (nr >= 0 && nr < ROWS && nc >= 0 && nc < COLS && map[nr][nc] != '#')
        {
        pr = nr; pc = nc;
    }
    if (map[pr][pc] == 'E') {              /* 胜利判定 */
        printf("You win!\n");
        running = 0;
    }
    Sleep(120);                             /* 控制帧率 */
}
```

14.2.3 设计要点

- **先判断、后移动**：所有移动都要经过”边界 + 是否可走”的双重检查，否则玩家会走出数组、踩穿墙。
- **逻辑与显示分离**：Render 只负责画，Update 只负责改数据。这样换界面、加道具都不必动游戏规则。
- **坐标顺序**：二维数组是 map[行][列]，屏幕上是 (x= 列，y= 行)，两者容易整体写反，导致”按上键往左走”。

14.3 Flappy bird 游戏

14.3.1 用物理量描述” 飞”

这类游戏的核心只有一条：**小鸟受重力，按键给它一个向上的速度**。把小鸟的纵向位置、纵向速度作为状态，每帧更新：

```
double y = 10.0;         /* 小鸟的纵向位置（行，可为小数以便平滑） */
double vy = 0.0;         /* 纵向速度 */
const double G = 0.6;    /* 每帧重力加速度 */
const double JUMP = -4.0;  /* 按键给的速度冲量（向上为负） */

/* 每一帧： */
vy += G;                 /* 先受重力 */
y  += vy;                /* 再按速度移动 */
if (y < 0 || y >= ROWS) { /* 撞上下边界，游戏结束 */ }
/* 玩家按键时： */
vy = JUMP;
```

为什么位置用浮点？因为每帧的位移可能不足一格，用整型会把小位移”吃掉”，动作会一顿一顿。显示时再取整即可。

14.3.2 障碍物与碰撞检测

障碍物通常用**结构体数组**描述（呼应第 12 章）：

```
typedef struct {
    int x;          /* 障碍物所在列，每帧减 1 向左移动 */
    int gapTop;     /* 空隙上沿的行号 */
    int gapHeight;  /* 空隙高度 */
    int scored;     /* 是否已计过分，防止重复加分 */
} Pipe;
Pipe pipes[MAXPIPES];
```

碰撞检测分两步：**先看横向是否重叠**（小鸟的列是否落在障碍物的列范围内），**再看纵向是否落在空隙之外**。只有横向重叠时才需要做纵向判断，这是最省事的写法：

```
for (i = 0; i < MAXPIPES; i++) {
    if (pipes[i].x == birdCol) {                        /* 横向重叠 */
        if (birdRow < pipes[i].gapTop ||                /* 撞上管 */
            birdRow > pipes[i].gapTop + pipes[i].gapHeight) {  /* 撞下管 */
            gameOver = 1;
        }
    }
    if (!pipes[i].scored && pipes[i].x < birdCol) {     /* 越过即加分 */
        score++; pipes[i].scored = 1;
    }
}
```

14.3.3 游戏的状态机

一个完整的游戏通常有三个状态，用枚举表示（呼应第 12 章）、用 `switch` 分派（呼应第 5 章）：

```
enum state { READY, RUNNING, OVER };
enum state st = READY;
/* 主循环里 */
switch (st) {
    case READY:   /* 显示提示，等按键开始 */      break;
    case RUNNING: /* 物理更新、障碍移动、碰撞检测 */ break;
    case OVER:    /* 显示分数，等按键重开 */        break;
}
```

把”开始、进行、结束”显式建模，比用一堆散布各处的 if 标志清楚得多，也更容易加重开、暂停、难度递增。

14.3.4 模块划分

表 14.2: 游戏程序的典型模块

模块	职责
`Init`	初始化地图/障碍物/分数/随机种子
`ReadInput`	非阻塞读键，转成” 动作”
`Update`	按规则更新所有状态（物理、移动、生成障碍）
`CheckCollision`	边界与碰撞判定，判定胜负
`Render`	清屏、绘制地图与实体、显示分数

> **核心考点** 游戏程序的 bug 大多不在” 算法”，而在” 状态没管干净”：某个变量在重开时没复位、障碍物越界后没有回收、加分判断重复触发。写完每个函数都问一句” 它读写哪些状态变量、这些变量在什么时机会变”，比盯着屏幕找现象有效。

14.4 本章知识点小结

- 动画 = 视觉暂留 + 帧循环；一帧 = 读输入、更新状态、清屏重绘、延时。
- 控制台动画的四个手段：清屏、光标定位、延时（毫秒）、非阻塞按键；双缓冲可消除闪烁。
- 迷宫用二维字符数组表示地图，用行列坐标表示玩家；移动前必须做边界检查与撞墙判断。
- 物理类游戏用” 位置 + 速度 + 加速度” 逐帧递推；位置用浮点，显示时再取整。
- 障碍物用结构体数组组织；碰撞检测宜” 先横向、后纵向”，减少无效判断。
- 用枚举描述游戏状态（就绪/进行/结束），用 `switch` 分派，结构最清晰。
- 按 `Init/ReadInput/Update/CheckCollision/Render` 分模块，逻辑与显示分离。
- 本章把全书知识串了起来：数组（地图）、指针与结构体（实体）、函数（模块）、循环与分支（规则）、文件（存档与最高分）。

14.5 本章常见错误小结

1. **用阻塞式输入：** `getchar()` 卡住帧循环，画面停住不动；应改用 `kbhit()` 配合 `getch()`。
2. **忘记清屏或清屏方式不当：** 每帧都 `system("cls")` 会明显闪烁，可改为光标定位、只重画变化处。
3. **Sleep 的单位搞错：** `Sleep(1)` 是 1 毫秒（几乎看不见），想要” 每帧 0.1 秒” 应写 100。
4. **移动前不做边界检查：** 玩家走出数组范围，读到随机内存，表现为” 地图突然乱掉” 或崩溃。
5. **行列颠倒：** `map[x][y]` 与 `map[y][x]` 混用，导致上下与左右方向错乱。

6. **浮点位置直接当地址用**：未经取整就用作数组下标，可能截断或越界。
7. **加分/碰撞重复触发**：没有” 已处理” 标志，一次通过被记多次分，或死后仍继续判定。
8. **重开游戏忘记复位状态**：分数、位置、障碍物沿用上一局的残值，越玩越乱。
9. **随机数没设种子**：每次运行障碍物位置完全相同；应在 `Init` 中调用一次 `srand(time(NULL))`。